

Heterogeneous Multi-robot Task Allocation for Long-Endurance Missions in Dynamic Scenarios

Álvaro Calvo and Jesús Capitán

Abstract—We present a framework for Multi-Robot Task Allocation (MRTA) in heterogeneous teams performing long-endurance missions in dynamic scenarios. Given the limited battery of robots, especially for aerial vehicles, we allow for robot recharges and the possibility of fragmenting and/or relaying certain tasks. We also address tasks that must be performed by a coalition of robots in a coordinated manner. Given these features, we introduce a new class of heterogeneous MRTA problems which we analyze theoretically and optimally formulate as a Mixed-Integer Linear Program (MILP). We then contribute a heuristic algorithm to compute approximate solutions and integrate it into a mission planning and execution architecture capable of reacting to unexpected events by repairing or recomputing plans online. Our experimental results show the relevance of our newly formulated problem in a realistic use case for inspection with aerial robots. We assess the performance of our heuristic solver in comparison with other variants and with exact optimal solutions in small-scale scenarios. In addition, we evaluate the ability of our replanning framework to repair plans online.

Index Terms—Multi-robot task allocation; heuristic planning; long-endurance missions; multi-UAV applications

I. INTRODUCTION

THE use of heterogeneous robot teams is rapidly expanding in applications that benefit from the combination of different robot capabilities, such as inspection [1], [2], agriculture [3], [4], or fire fighting [5]. For example, Unmanned Aerial Vehicles (UAVs) could be combined with ground robots [3], [6], as the former can access more distant places while the latter can carry heavier equipment. Cooperation among heterogeneous UAVs is another option [1], [5], [7], as they may provide different maneuverability (e.g., rotary vs. fixed-wing vehicles) or sensors and manipulation/delivery capabilities depending on the vehicle. In these applications, MRTA can become especially hard, as they usually pose a multi-objective optimization with multiple constraints; e.g., some tasks may only be executed by certain robots with the required capabilities, or some could need to be accomplished by multiple robots in a synchronous manner, among others. We are interested in long-endurance missions in outdoor environments. This setting brings two additional challenges: 1) battery capacities could be limiting for the robots, especially for multi-UAV teams, which inspire our work, so recharging operations should be scheduled during operation; and 2) these outdoor scenarios are typically dynamic and require online replanning to perform long-endurance missions robustly, potentially dealing with robot delays and failures.

This work was supported by project PID2024-161069OB-C33, funded by MICIU/AEI/10.13039/501100011033 and FEDER, UE.

The authors are with the Multi-robot & Control Systems group, Universidad de Sevilla, Spain, {acalvom, jcapitan}@us.es.

In this paper, our objective is to devise a planning framework to solve a new class of heterogeneous MRTA problems. We introduce recharge operations to extend robots' autonomy, and we endow the problem with greater flexibility by allowing certain tasks to be fragmented and/or executed by coalitions – with a non-fixed size – of robots, as well as allowing the possibility of having inter-robot relays. Moreover, as we aim for dynamic scenarios where unplanned events may occur, our framework is able to detect these events and repair or fully recompute plans in real time to adapt to the new circumstances. To the best of our knowledge, there is no alternative method in the literature that combines all these problem features (see related works in Section II). Therefore, we take a step forward by categorizing, formulating, and solving this novel class of heterogeneous MRTA problems for long-endurance missions. The contributions of this work are the following:

- 1) Motivated by heterogeneous teams of robots performing long-endurance missions in dynamic outdoor settings, we pose a novel MRTA problem (Section III) where robot recharges are allowed and different types of tasks are combined, depending on their level of decomposability and on the size flexibility of the robot coalition required to execute them. We discuss the categorization of our problem within well-known MRTA taxonomies in the literature and prove its NP-hard complexity.
- 2) We formulate our MRTA problem as a MILP. Our formulation is general enough to account for all problem features (Section IV), integrating heterogeneous robot capabilities, recharge operations, task decomposition, inter-robot relays, and multi-robot tasks performed by synchronized coalitions of fixed or variable size. From a theoretical perspective, this MILP formulation helps analyze the complexity and characteristics of the optimal solutions of our new class of MRTA problems.
- 3) Given the NP-hardness of the problem, exact solvers suffer from scalability issues to tackle the MILP posed here. Therefore, we contribute a novel heuristic algorithm that leverages specific properties of the problem in order to find approximate solutions that comply efficiently with all constraints (Section V).
- 4) To robustly cope with dynamic scenarios, we integrate our heuristic solver into a mission planning and execution framework (Section VI). Mission execution is monitored to 1) repair plans in case of robot delays; and 2) recompute the full plan online if a repair is not possible or an unexpected event occurs, such as a robot failure or the arrival of new tasks.

- 5) We provide extensive experimental results to demonstrate our algorithms in a realistic use case for multi-UAV inspection (Section VII). We compare our heuristic plans with the optimal solutions of the MILP formulation in small-scale scenarios. We then assess the performance of the heuristic solver for larger scenarios and evaluate our whole replanning and execution framework. All our code is provided open-source for the community and integrated into a ROS-based architecture.¹

This paper is an *evolved version* of our previous work [8], where we introduced our MILP formulation. We extend it with the heuristic algorithm to find approximate solutions efficiently and with the online replanning framework for dynamic scenarios. Additionally, we provide a full set of new experimental results and release all the code as open source.

II. RELATED WORK

A number of comprehensive MRTA taxonomies can be found in the literature [9]–[11]. The existing methods can be classed as “centralized,” where a single entity has access to all robots’ information to perform task allocation, or “distributed,” in which robots compute their plans locally and exchange information with others to converge to a common solution. They can also be grouped into exact methods, which provide optimal (or near-optimal) solutions, and methods that can provide approximate solutions more efficiently (typically using heuristics) and are more suitable for online task allocation.

A. Distributed approaches

Market-based approaches have been widely used for MRTA, such as auctions [12], [13], which involve auctioning tasks to robots through a bidding process based on a utility function that combines the robot’s capabilities and the problem constraints. Even though they can be centralized, these methods are typically implemented in a distributed fashion, where each robot determines its own bid for tasks through an internal cost function. Besides auctions, other distributed greedy methods, such as distributed versions of the Hungarian algorithm [14] and task-swapping algorithms [15], iterate through pairs of robots, exchanging tasks to improve team performance.

Auction-based methods rarely consider schedules and task ordering constraints. The work by Krizmancic et al. [6] is an exception, as they address temporal and precedence constraints for task allocation in aerial-ground robot teams for automated construction. Others have also proposed decentralized auctions for multi-robot task scheduling with time windows, considering either task precedences [16] or robot capacities [17].

Ferreira et al. [3] present a distributed algorithm for task scheduling, considering precedence constraints but not multi-robot tasks. They propose an MILP formulation (and a genetic solver) in which robots have heterogeneous capabilities and battery constraints, but do not model recharges. Another approach is to use probabilistic methods, as Smith et al. do [7], with a distributed algorithm based on a Monte Carlo tree search. They consider battery-limited robots but do not

address temporal constraints or multi-robot tasks. Generally, these distributed methods struggle to find optimal solutions in complex problems with tightly coupled restrictions, such as those targeted in this work.

B. Multi-agent planning and scheduling

The multi-agent planning community has devoted significant effort to solving task planning and scheduling problems [18]. In these scenarios, each robot’s plan consists of an ordered set of actions (or tasks) that can have varying durations and can be executed sequentially or in parallel, with strong time-related dependencies. Some works address uncertainty in task durations or outcomes [19], [20], but few focus on multi-robot tasks where a coalition performs the same task concurrently. For instance, by assuming minimum and maximum coalition sizes, multi-robot tasks can be decomposed into single-robot tasks solvable by classical single-agent planning, with solutions subsequently combined [21]. A separate collaborative action would be included for each possible combination of robots that can execute each multi-robot task. The limit on the number of coalition members has also been used to partition the problem into loosely and tightly coupled components and solve them separately, incrementally increasing the number of robots that share a task [22]. Unlike these approaches, our method does not require fixing intervals for the number of robots in a task, which allows for greater flexibility. Additionally, in contrast to these prior approaches, we prioritize planning for recharges over task ordering constraints.

C. MILP formulations

Operations research offers a variety of related problem formulations. Arc Routing Problems (ARPs) involve covering a set of graph edges (arcs) with one or more vehicles [23]. The Capacitated Arc Routing Problem (CARP) adds limited vehicle capacities per tour, suitable for modeling battery-limited robots. In robotics, Vehicle Routing Problems (VRPs) and their variants [24] are more common. VRPs involve computing tours for one or more vehicles to visit spatial locations (graph nodes), starting and returning to one or more depots. VRPs generalize the Traveling Salesman Problem (TSP), where the vehicle must return to the initial depot. These NP-hard problems are often addressed with heuristic methods due to the limitations of MILP formulations.

Agarwal et al. [2] proposed a CARP for power line inspection with multiple UAVs, modeling directional and state-dependent (inspecting vs. traveling) edge costs, such as wind effects. Vehicle battery life is limited per tour, but recharges are not considered. While many ARP/VRP formulations incorporate battery constraints or heterogeneous capabilities, recharging operations and inter-robot synchronization for multi-robot tasks are rare. For instance, Dorling et al. [25] presented a UAV delivery VRP with vehicle reuse (recharging), including deadlines but not multi-robot tasks. Li et al. [26] proposed a multi-period MILP formulation (without time constraints) to model recharges in a multi-UAV traffic monitoring CARP variant.

¹Robot Operating System, <https://www.ros.org>.

Planning recharge times and locations for *persistent* UAV teams has garnered significant interest due to its relevance to aerial monitoring and surveillance. Several works [27]–[31] have presented TSP variations that provide schedules specifying when and where UAVs recharge between tours. Mathew et al. [27] consider multiple moving ground recharging stations, discretizing UAV trajectories into projected ground points for a graph-based abstraction. Ding et al. [28] also consider mobile depots, formulating a generalized multiple depot TSP for persistent UAV surveillance. Yu et al. [29] plan recharge-inclusive tours for a single UAV, optimizing visit order, recharge times, and locations. Diller et al. [30] propose a mixed-integer nonlinear program for joint path planning of a UAV and a moving ground vehicle, allowing the UAV to adjust its speed between rendezvous for recharging. Maini et al. [31] minimize coverage time by optimizing routes and rendezvous locations for a UAV-ground vehicle team. Arribas et al. [32] address persistent aerial service, where each location requires continuous UAV coverage, minimizing fleet size while ensuring persistent service with recharges. While these works focus on persistent tasks, our work addresses a broader problem encompassing fragmentable and relayable multi-robot heterogeneous tasks.

Another important aspect of the problem addressed in this work is the existence of temporal constraints and multi-robot tasks; i.e., tasks that have to be executed by several robots working in a synchronized fashion. A first step to tackle this is forming coalitions made up of robots with heterogeneous capabilities, which collectively fulfil the task requirements. For instance, Ramchurn et al. [33] propose an MILP formulation where coalitions consist of robots with the same capabilities but different degrees of effectiveness performing multi-robot tasks with time deadlines. Gosrich et al. [34] developed a mixed integer non-linear programming approach to formulate a MRTA problem with task precedence constraints and robot coalitions for multi-robot tasks. A second step is to deal with the temporal constraints. For this, we focus on task deadlines and robot synchronization for multi-robot tasks and relayable tasks. Bredstrom et al. [35] proposed a VRP with time windows where some visits must be pairwise synchronized due to application requirements. Instead of minimizing waiting times as we do, they formulate hard constraints that impose an equal arrival time on certain robots performing different tasks. Flushing et al. [36] presented an MILP for heterogeneous MRTA, where *non-atomic* tasks are considered; i.e., tasks that can be executed incrementally over disjoint periods of time. This is related to our concept of fragmentable tasks, which are tasks split into several segments, though we also add the possibility of having relayable tasks, which implies additional synchronization restrictions between the robot leaving the task and the one picking it up. These works do not typically consider reusable robots with limited operation time that can be reused by means of periodic recharges. Overall, among all existing MILP formulations in the state of the art, we believe that there is a gap consisting of methods combining all the features addressed in this work. Either multi-robot, fragmentable, and relayable tasks are not considered, or vehicle recharges are not modeled.

D. Heuristics

Due to the high complexity of the multi-robot optimization problems discussed in this work, to cope with either heterogeneous robot capabilities or multi-robot and fragmentable tasks with temporal constraints, finding optimal solutions is usually computationally demanding, even prohibitive in certain cases. Therefore, heuristic algorithms are commonly proposed to compute approximate solutions in a more efficient and scalable manner.

Metaheuristic algorithms are a first approach; tabu search, genetic algorithms, and simulated annealing being the most common options for MRTA problems. For instance, genetic algorithms have been applied successfully to solve problems with heterogeneous robots and tasks with temporal and precedence constraints [3], [37]–[39]. Simulated annealing has been used to solve delivery problems where UAVs can make multiple trips by recharging [25], and recently, to solve scheduling problems for multi-robot manufacturing [40]. The metaheuristic Variable Neighborhood Search (VNS) has also been used for orienteering problems in multi-UAV data collection applications [41], [42].

On the one hand, metaheuristics have the advantage that they are generic algorithms that can be adapted to encode many diverse problems. On the other hand, as they rely on random search, they can require a large amount of computation time to identify promising solutions. Another option is to use heuristics specifically designed for the application at hand, trying to leverage any a priori known information about the problem structure. In this category, it is worth noting some heuristics for well-known problems in the literature that are related to ours. For instance, the Lin-Kernighan-Helsgaun (LKH) heuristic [43], which follows a strategy based on local search, has been widely used for different variants of the TSP, including problems that consider UAV recharges [27], [31]. Agarwal et al. [2] proposed a novel heuristic for a multi-UAV CARP problem. The underlying idea is to create an initial set of tours (one per arc to visit) which are then merged together in a greedy fashion to improve the solution. Al-Hussaini et al. [44] discussed existing heuristics for scheduling problems, and proposed a new one for a problem with multi-robot tasks and temporal constraints. Gombolay et al. [45] presented another heuristic algorithm supporting temporal windows and precedence constraints for the tasks, but also heterogeneous robot capabilities. They use a multi-agent task sequencer, inspired by real-time processor scheduling techniques, in conjunction with an MILP solver that resolves task-robot allocation. Although they outperform other relevant state-of-the-art heuristic solvers, yielding nearly-optimal solutions, battery-limited robots and multi-robot tasks are not modelled. Messing et al. [46] contributed a unified framework for task planning and scheduling in heterogeneous multi-robot teams. Their solution is a multi-layer approach that interleaves task planning, allocation, scheduling, and motion planning until a valid plan is found, applying heuristic search-based algorithms at the different layers. The framework is quite generic and includes task temporal constraints and multi-robot tasks performed by robot coalitions (they add wait constraints

to synchronize those coalitions). However, they do not cover battery constraints and recharging operations. Ramchurn et al. [33] did not cover battery constraints either, but they devised an interesting heuristic procedure to solve a problem where coalitions of heterogeneous robots have to be allocated to multi-robot tasks with deadlines.

Finally, reinforcement learning is a promising alternative to heuristic methods for addressing the computational burden of MRTA [47], [48]. However, key challenges with these methods include their lower interpretability and lack of optimality guarantees.

E. Replanning approaches

Apart from the traditional methods for *offline* task allocation, there are specific frameworks in the literature for dynamic MRTA, which monitor task execution during operation and trigger some replanning procedure if the running plan is not valid anymore after an unexpected event has occurred (e.g., a change in some task requirement or robot capabilities). The idea is to reallocate incomplete tasks not from scratch but taking into account the previous plan and the effects of the disruptive event on task performance.

Al-Hussaini et al. [44] presented a heuristic algorithm that provides reallocation suggestions to handle contingencies during mission execution, such as a robot failing, a new task arriving, a task changing its parameters, a task being identified as risky, or other events of this nature. They consider multi-robot tasks with deadlines and precedence constraints as well as the possibility of creating additional rescue and relay tasks, the former to rescue a disabled robot, the latter to send a robot close enough to a subgroup of teammates out of communication range in order to share a new plan with them. Neville et al. [49] also proposed an approach for dynamic task allocation in heterogeneous multi-robot settings with task temporal constraints. Multi-robot tasks are performed by robot coalitions that are created using a trait-based method. Overall, their method interleaves task allocation, scheduling, and motion planning in order to compute solutions incrementally by performing graph-based heuristic searches. After an unexpected event, a new solution is efficiently computed by repairing only the necessary nodes in the task graph. Leahy et al. [4] developed a framework for task planning in teams of robots with heterogeneous capabilities. They define a specification language based on temporal logic, which a user can employ for high-level mission specification, allowing temporal constraints and multi-robot tasks. These specifications are then encoded in an MILP formulation whose objective is optimizing robustness against robot failures. Thus, the plan computed is the one that tolerates the largest number of robot dropouts. Online replanning is enabled by means of another MILP that modifies the original after a robot dropout. Recently, [50] presented an MILP formulation for task allocation in human multi-robot teaming scenarios. After the mission has started, execution of the plan is continuously monitored, and a replanning procedure is triggered if the current plan is no longer feasible or if its expected quality (according to the application cost function) has decreased below a certain

threshold. Unlike ours, these methods do not consider battery-limited robots with the possibility of recharging, although the concept of creating *virtual* robot rescue tasks [44] may be similar to our recharge tasks.

III. PROBLEM DESCRIPTION

We address a task planning problem with a team of heterogeneous cooperating robots. The robots are heterogeneous in the sense that they can provide different sensing/locomotion capabilities, and not all of them are suitable for every task. For instance, a robot with the ability to manipulate and transport items will be suitable for a delivery task, while a robot with a specific camera onboard could be required for a particular inspection task. Each robot has a limited operational time given by its battery capacity, but unlimited recharges are allowed (with an associated time cost) at fixed stations with known positions to reset the robot's battery level. Each task has a known spatial location and a predefined time duration; we do not have specific time windows in which tasks need to start their execution, nor precedence constraints, but only a maximum completion time for each task (*deadline*), as a way to establish different priorities between tasks.

The goal is to compute the optimal plan (minimizing the *makespan*; i.e., the completion time of the whole mission) for the team, given all constraints. This means devising a schedule containing the set of ordered tasks that each robot has to execute and their start time instants. Moreover, we consider dynamic scenarios, in the sense that new tasks may arrive at any time and robots may fail when executing an assigned task or while traveling. This implies the need for online replanning in order to react to new tasks or circumstances.

One of the major novelties of this work is given by the complexity of the allocation problem that we tackle, as we consider different types of tasks that yield a new task categorization. Before describing these task categories, let us propose a running example that will be used to motivate our work and to better illustrate the different types of tasks.

Running example: *Our main focus in this work is the use of heterogeneous teams of UAVs to execute long-endurance missions in outdoor settings. In these scenarios, the limited flight time of UAVs is key, and this is why we explicitly allow recharging operations when planning, to permit extended periods of autonomy. Thus, imagine an application where a team of UAVs provides support to human workers during inspection operations in a solar energy plant. Depending on their capabilities, the UAVs could be sent to inspect remote areas of the plant, to monitor worker operations for safety issues, to deliver tools or other items to some workers, and so on. One or several base stations would be installed at known positions around the plant so that the UAVs can recharge when needed. Although there may be a starting set of scheduled tasks, as the inspection mission evolves, human operators could decide to order new tasks to be assigned to the supporting UAVs, which makes the mission dynamic. Moreover, due to unexpected situations or hardware issues, UAVs could run out of battery and become unavailable.*

Now let us describe the task categorization that we consider in our problem. As shown in Figure 1, tasks are classified ac-

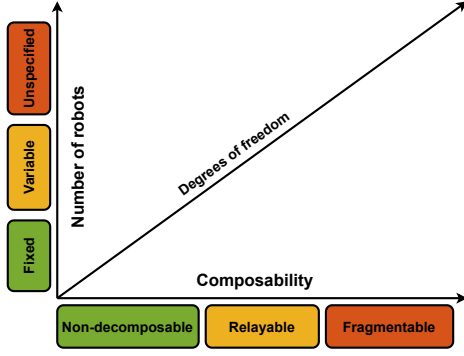


Figure 1: Complexity of the problem according to task categorization. Tasks are classified according to their decomposability and coalition size flexibility; a higher degree of freedom implies a planning problem that is harder to solve.

cording to two properties that determine the overall complexity of the problem: task decomposability and task coalition size flexibility.

- **Decomposability:** This property indicates the capacity of a task to be decomposed into subtasks. We consider three types: *non-decomposable*, *fragmentable* and *relayable*. Non-decomposable tasks have to be executed entirely by the same set of robots from start to end without pause. Fragmentable tasks can be divided into a set of fragments (each fragment duration is obtained by also dividing the original task duration) that can be executed independently by different robots. There are no dependencies between the timelines of the various fragments: they could be executed in any order, with gaps in between, and even overlapping in time. In contrast, relayable tasks can be divided into a sequence of fragments that must be executed on a continuous timeline without gaps in between. This division is made when the assigned robot (or multi-robot coalition) does not have enough battery to perform the entire task and there is a need for relays. Therefore, in relayable tasks, consecutive fragments are executed by different robots, as robots leaving to recharge are replaced by new ones. In our running example, a non-decomposable task could be a UAV that has to deliver a tool to an operator; this task has to be carried out entirely by the same robot without interruptions. A fragmentable task could be the aerial inspection of a particular area of the solar plant to search for malfunctioning elements. In this case, the inspection area could be divided into several parts, and these could be inspected independently. For example, the same UAV could inspect all subareas (task fragments) sequentially, even including recharges in between; or the subareas could be inspected by different UAVs in parallel. A relayable task could be a UAV that has to use its onboard camera to monitor a worker operating in the field for safety issues. When the operation is risky, it is critical to have a continuous video stream of the worker. Therefore, if the task duration is too long, successive UAVs will have to relay each other as they run out of battery, in such a way that one is always monitoring

the operation.

- **Coalition size flexibility:** We consider multi-robot tasks that need to be jointly executed by a coalition of robots. This property refers to the flexibility of the task with respect to the required size of the coalition. We consider three types of specification for the coalition size. First, for tasks where the coalition size is *fixed* (it could be 1 for single-robot tasks) as a hard constraint, these tasks must be executed by coalitions with exactly the specified size. Second are tasks where the coalition size is *variable* as a soft constraint. This means that an ideal coalition size for the task is specified, but coalitions of different size are also allowed although with a penalty. The third kind is tasks where the coalition size is *unspecified*. These are tasks without constraints on the coalition size. Any coalition size is allowed without penalty and the task duration will depend on the final number of robots allocated. We assume that unspecified tasks are also fragmentable, which is usually the case, but not the opposite. Coming back to our running example, a task to monitor a worker's operation in the field could be requested to be executed by three UAVs simultaneously, but it may still be acceptable to execute it with fewer UAVs if there are not enough resources. This would be an example of a variable coalition size. In contrast, some inspection techniques for solar panels require the explicit cooperation of a fixed number of several UAVs. For example, photoluminescence inspection involves illuminating a specific portion of the solar panel surface and recording the luminescence emission generated in the remaining area. This could be done by a coalition of exactly two UAVs, one illuminating and the other acquiring images with a specialized camera onboard. An example of a task with an unspecified coalition size is a survey of a given area for standard thermal inspection. In this case, the more UAVs assigned, the shorter the task duration.

We cover what are known as ST-MR-TA problems according to a well-known taxonomy of MRTA [9]. This means that our robots are *Single-Task* (ST); they can only perform one task at a time. The tasks are *Multi-Robot* (MR); they could require multiple robots to be executed. Lastly, we have a *Time-extended Assignment*; i.e., each robot is allocated several tasks that must be executed according to a given schedule. Nunes et al. [10] extended this taxonomy to differentiate between problems with *Task Windows* (TW) and *Synchronization and Precedence* (SP) constraints. Our problem falls within the TW category, since we have no constraints on the start times for the tasks but we do have deadlines. Although we do not include precedence constraints between tasks, note that time synchronization is still imposed when multiple robots need to perform a task together or execute relays. Lastly, another well-known taxonomy [11] distinguishes between different types of problems depending on the inter-task relationship. Our decomposable tasks fall into the *Complex Dependencies* (CD) category defined in this taxonomy. Given that we have multi-robot tasks, there are inter-schedule dependencies, as plans

for each robot cannot be computed independently. However, CD problems imply an additional complexity, as the optimal decomposition of tasks must be computed jointly with the task allocation. This is our case, since some tasks may be split for recharges and there is an additional degree of freedom to decide when to recharge.

Finally, note that we do not allow partial execution of the tasks. All tasks must be executed completely in order for a mission plan to be valid. Since it would need to be decided to what extent each task is covered, partial execution would significantly increase the problem complexity without a clear advantage in practical scenarios (we can still propose complete solutions and replan after partial execution caused by a robot failure). Furthermore, *persistent* tasks are not explicitly considered either. These are tasks without a finite duration to be executed indefinitely until mission termination. For instance, monitoring a given perimeter with UAVs for security. Nonetheless, note that if we include a persistent task along with others that have deadlines, the persistent task would have the lowest priority. This means it could easily be added to the robots' plan when they become idle after completing their finite tasks.

A. Problem complexity

Theorem 1. *The heterogeneous MRTA problem proposed in this section is NP-hard.*

Proof. Given its additional flexibility in terms of task decomposability and coalition size, the problem posed here is a generalization of other well-known MRTA problems that are NP-hard. This can be proved by contradiction: let us assume the problem is not NP-hard. But if we can find a particular instance of our problem that is NP-hard, the overall problem must also be NP-hard, as there would not be a known algorithm to solve all its instances in polynomial time. Let us consider a specific instance of the problem where all robots have unlimited battery and capabilities to execute all tasks. Let all tasks be non-decomposable, with an arbitrarily large deadline and with a fixed coalition size of 1; i.e., all tasks are single-robot. In that case there is no need for recharges, fragmentation or multi-robot synchronization, and the problem consists of assigning the tasks to the different robots, which can start or end at different base stations. This would be equivalent to the well-known multi-depot vehicle routing problem, which is NP-hard [51]. $\square$

IV. MILP FORMULATION

In the following, we develop our mathematical formulation to solve the problem in Section III, cast as an MILP. Table I contains a list of symbols used and their descriptions.

A. Preliminary definitions

Let $\mathcal{R}$ be a set of n heterogeneous robots; each robot $r \in \mathcal{R}$ has an initial position $p_{r,0} \in \mathbb{R}^3$, a traveling speed v_r , a maximum battery time B_r^{max} (battery autonomy), and an initial battery time consumed $B_{r,0}$. Let $\mathcal{T}$ be the set of m tasks to be executed by the team; each task $t \in \mathcal{T}$ has a

Table I: Table of symbols separated by category. Within the MILP variables, decision variables are shown in bold.

Robot parameters	
$\mathcal{R}$	Set of n heterogeneous robots
$p_{r,0}$	Initial position of robot r
v_r	Traveling speed of robot r
B_r^{max}	Maximum battery time of robot r
$B_{r,0}$	Initial battery time consumed of robot r
B_r^{min}	Minimum remaining safety battery time for robot r
Task parameters	
$\mathcal{T}$	Set of m tasks to be allocated
$\tilde{\mathcal{T}}$	Set of tasks including recharge task
$\mathcal{T}_0$	Set of n auxiliary tasks to encode initial robot positions
p_t	Spatial location of task t
T_t^e	Estimated execution time for task t
T_t^{max}	Deadline to complete task t
T_{r,t_1,t_2}^d	Displacement time from task t_1 to t_2 for robot r
N_t	Coalition size specified for task t
t_R	Recharge task
$H_{r,t}$	Capability (binary) of robot r to execute task t
Problem parameters	
$\mathcal{S}$	Set of time slots for each robot's queue
N_f	Maximum number of fragments for a task
$\eta_1, \eta_2, \eta_3, \eta_4$	Normalization constants for the objective function
MILP variables	
$x_{r,t,s}$	It encodes if task t is allocated to slot s of robot r
n_t^f	Number of fragments in which task t is divided
n_t	Number of times task t appears among all robot queues
n_t^q	Number of robot queues where task t appears
n_t^r	Number of robots executing task t simultaneously
$n_{r,t}^q$	Variable encoding if task t appears in robot r 's queue
$T_{r,s}^w$	Waiting time for robot r in slot s
$T_{r,s}^d$	Displacement time for robot r to reach task in slot s
$T_{r,s}^e$	Execution time for task allocated to slot s of robot r
$T_{r,s}^f$	Finish time for task allocated to slot s of robot r
$B_{r,s}$	Battery time consumed by robot r at the end of slot s
y_{t,r_1,s_1,r_2,s_2}	It encodes if robots r_1 and r_2 need to synchronize task t in slots s_1 and s_2 , respectively
z_{t,r_1,s_1,r_2,s_2}	It encodes if robots r_1 and r_2 need to relay task t in s_1 and s_2 , respectively
Z	Makespan of the mission
V_t	Deviation in the specified coalition size for task t
$\Delta T_{r,s}^{max}$	Delay for robot r to complete task in slot s
Heuristic planner variables	
Q	List with the queue of tasks assigned to each robot
$\tilde{\mathcal{T}}$	List with all task fragments to be allocated
M_R	Binary matrix indicating robot pre-recharges
$\mathcal{R}^c$	Set of compatible robots with a task
$\mathcal{R}_t, \tilde{\mathcal{R}}_t$	Best and auxiliary robot coalitions for task t
B_t	Lower bound for the remaining battery time of robots compatible with task t
n_t^c	Number of robots compatible with task t
n_t^e	Number of excess compatible robots for task t
f_t	Recharge pattern frequency for task t

spatial location p_t , an estimated execution time T_t^e , a deadline to complete the task T_t^{max} , and a number of robots needed N_t (coalition size).² Apart from the actual tasks, we include an additional task t_R so that the robots can recharge their batteries at one of the available base stations, defining $\tilde{\mathcal{T}} = \mathcal{T} \cup t_R$. Our formulation is agnostic to the method of selecting the *best* station to recharge at each point in time, and we assume a fixed execution time $T_{t_R}^e$ to fully recharge batteries.

The heterogeneous capabilities of the robots are encoded through a set of binary variables $H_{r,t} \in \{0,1\}$, where $H_{r,t} = 1$ if robot r has the hardware required to execute

²Note that this requirement could be hard or soft depending on the coalition size flexibility of the task.

task t , and 0 otherwise. For each robot–task pair, we define a displacement time T_{r,t_1,t_2}^d , which is the estimated time to navigate robot r from the location of task t_1 to the location of task t_2 . We compute this navigation time using Euclidean distances between tasks (which may be available from a topological map) and the speed of each robot v_r . In order to model the initial positions of the robots, $t_1 \in \tilde{\mathcal{T}} \cup \mathcal{T}_0$ and $t_2 \in \tilde{\mathcal{T}}$, where $\mathcal{T}_0$ represents a set of n auxiliary fictitious tasks, each located at the initial position of each robot $p_{r,0}$.

To model the decision variables, we need to decide which tasks are allocated to each robot and in which order. For this, we introduce the concept of time slots: each robot has a task queue made up of a series of slots of variable duration (let $\mathcal{S}$ be the set of slots for each queue), where tasks can be allocated. The binary decision variables $x_{r,t,s} \in \{0,1\}$ take value 1 if task t is assigned to slot s of robot r , and 0 otherwise. The duration of each slot will depend on the time required by the specific task placed in that slot, so slot durations will differ among robots, depending on their task assignment. In general, the sizes of the robot schedules (number of assigned tasks) can differ; however, for implementation purposes, all robot queues have the same size $|\mathcal{S}|$, and only the necessary slots are *activated* for each robot through the variables $x_{r,t,s}$. Furthermore, since decomposable tasks can be split into several fragments, we also define the number of fragments into which each task is divided $n_t^f \in \mathbb{N}$, $n_t^f \in [1, N_f]$, where N_f is the maximum number of fragments into which any task can be divided.³ Given the maximum battery time for robots, we can compute a bound for N_f by considering the worst case of the longest task and check the number of tours that would be needed (each tour implies a new fragment and a recharge).⁴ Note that each fragment is allocated to a different robot slot and its execution time is computed as T_t^e/n_t^f .

B. Basic constraints

Some basic constraints must hold regarding the assignment of tasks to slots:

$$\sum_{t \in \tilde{\mathcal{T}} \cup \mathcal{T}_0} x_{r,t,s} \leq 1, \quad (1a)$$

$$x_{r,t_R,s-1} + x_{r,t_R,s} \leq 1, \quad (1b)$$

$$\sum_{t \in \tilde{\mathcal{T}}} x_{r,t,s} \leq \sum_{t \in \tilde{\mathcal{T}}} x_{r,t,s-1}, \quad (1c)$$

$$\forall r \in \mathcal{R}, s \in \mathcal{S}.$$

(1a) ensures that there is no more than one task per slot; (1b) avoids solutions with consecutive recharges for the same robot; and (1c) prevents the existence of free slots between tasks in a queue; tasks should occupy the lowest possible slots in the queue and there should be empty slots after the last assigned task. Moreover, there must be hardware compatibility for the robots assigned to a task:

$$x_{r,t,s} \leq H_{r,t}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T}, s \in \mathcal{S}. \quad (2)$$

³This variable is forced to be 1 for recharges and non-decomposable tasks.

⁴A similar method is used to compute a valid value for $|\mathcal{S}|$.

Additionally, we define some auxiliary variables for counting: $n_t \in \mathbb{N}$ counts the total number of times task t appears among all robot queues, $n_t^q \in \mathbb{N}$ counts the number of queues in which task t appears, and $n_t^r \in \mathbb{N}$ is the number of robots executing task t simultaneously (this is the coalition size and takes value 1 for single-robot tasks). $n_{r,t}^q \in \{0,1\}$ is a binary variable that takes the value 1 if task t appears in the queue of robot r . Formally,

$$n_t = \sum_{r \in \mathcal{R}} \sum_{s \in \mathcal{S}} x_{r,t,s}, \quad (3a)$$

$$n_t^q = \sum_{r \in \mathcal{R}} n_{r,t}^q, \quad (3b)$$

$$n_t^r \leq n_t^q, \quad (3c)$$

$$n_t^r \geq 1, \quad (3d)$$

$$n_t = n_t^r \cdot n_t^f, \quad (3e)$$

$$\forall t \in \mathcal{T}.$$

Note that (3d) means that all tasks are assigned to at least one robot, which makes sense if we assume that the multi-robot team has the required hardware and size to accomplish all tasks in the scenario, and that the battery autonomy of the robots is enough to reach each task, execute it, and return to a base station.

C. Time-related variables

We define $T_{r,s}^d$ as the time required to move robot r to the location of the task allocated to slot s , starting at the location of its previous task, assigned to slot $s-1$; $T_{r,s}^e$ as the time required to execute the task allocated to slot s ; and $T_{r,s}^f$ as the time when the task allocated to slot s finishes. We also define $T_{r,s}^w$ as the time that robot r has to wait in slot s before starting its allocated task, to coordinate task execution with other robots. Recall that we consider multi-robot tasks, where time synchronization must be enforced so that all the robots involved start the task simultaneously. A similar synchronization is needed when relaying takes place in a relayable task. This is done by establishing this *waiting* time for each robot before starting, so that those arriving earlier wait for the others. The value of the waiting time will depend on the arrival time of all the robots involved. More formally,

$$T_{r,s}^d = \sum_{t_2 \in \tilde{\mathcal{T}}} \left(\sum_{t_1 \in \tilde{\mathcal{T}} \cup \mathcal{T}_0} T_{r,t_1,t_2}^d \cdot x_{r,t_1,s-1} \right) \cdot x_{r,t_2,s}, \quad (4a)$$

$$T_{r,s}^e = \sum_{t \in \tilde{\mathcal{T}}} (T_t^e/n_t^f \cdot x_{r,t,s}), \quad (4b)$$

$$T_{r,s}^f = T_{r,s-1}^f + T_{r,s}^d + T_{r,s}^w + T_{r,s}^e, \quad (4c)$$

$$T_{r,0}^f = 0, \quad (4d)$$

$$\forall r \in \mathcal{R}, s \in \mathcal{S}.$$

$B_{r,s}$ is the battery time accumulated by robot r at the end of slot s , which is computed recursively, with an initial value $B_{r,0}$, (5a) taking into account that the waiting and execution

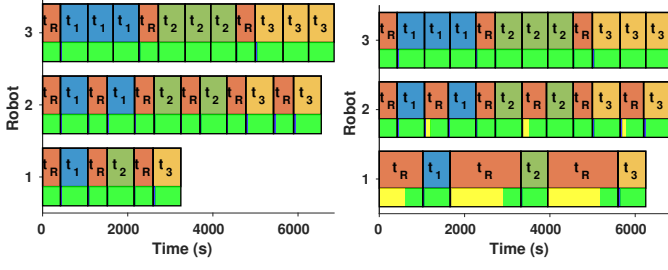


Figure 2: Example with 3 robots executing 3 consecutive multi-robot relayable tasks (with different color code). For each task, displacement time is depicted in blue, waiting time in yellow and execution time in green. Each task is divided into 3 fragments ($n_t^f = 3$) executed by coalitions of 2 robots ($n_t^r = 2$). Robot 3 executes all tasks and recharges in between, while robots 2 and 1 relay each other to accompany robot 3. Left, solution where robots do not coordinate task execution; right, solution including time coordination constraints.

times during recharge tasks do not consume battery.⁵ Since robot battery autonomy is limited, (5b) constrains the battery time consumed up to any slot to be no greater than the maximum battery time available, always leaving a minimum battery time B_r^{min} available for safety reasons. Robots should be able to go back to a recharge station with that amount of battery time.

$$B_{r,s} = B_{r,s-1} \cdot \bar{x}_{r,t_R,s-1} + T_{r,s}^d + (T_{r,s}^w + T_{r,s}^e) \cdot \bar{x}_{r,t_R,s}, \quad (5a)$$

$$B_{r,s} \leq B_r^{max} - B_r^{min}, \quad (5b)$$

$$\forall r \in \mathcal{R}, s \in \mathcal{S},$$

where $\bar{x}_{r,t,s} = 1 - x_{r,t,s}$.

D. Time coordination

Robots must coordinate their timing while executing a task in two different situations: when several robots need to perform a multi-robot task together or when a relay is carried out. Figure 2 shows an example to illustrate the difference between solutions when the schedules are coordinated and when they are not. This time coordination requires additional constraints which we model with two types of binary decision variables: $y_{t,r_1,s_1,r_2,s_2}, z_{t,r_1,s_1,r_2,s_2} \in \{0, 1\}$. $y_{t,r_1,s_1,r_2,s_2} = 1$ indicates that t is a multi-robot task assigned to slot s_1 of robot r_1 and slot s_2 of robot r_2 and that the two robots must be synchronized; and $z_{t,r_1,s_1,r_2,s_2} = 1$ indicates that a fragment of task t assigned to slot s_1 of robot r_1 is relayed by another fragment of the same task assigned to slot s_2 of robot r_2 . Then robot time coordination is enforced by:

$$T_{r_1,s_1}^f \cdot y_{t,r_1,s_1,r_2,s_2} = T_{r_2,s_2}^f \cdot y_{t,r_1,s_1,r_2,s_2}, \quad (6a)$$

$$T_{r_1,s_1}^f \cdot z_{t,r_1,s_1,r_2,s_2} = (T_{r_2,s_2}^f - T_{r_2,s_2}^e) \cdot z_{t,r_1,s_1,r_2,s_2}, \quad (6b)$$

$$\forall r_1 \neq r_2, r_1', r_2' \in \mathcal{R}, s_1, s_2 \in \mathcal{S}, (r_1', s_1) \neq (r_2', s_2), t \in \mathcal{T}.$$

⁵If slot s represents a recharge, $B_{r,s}$ is not zero but the accumulated battery time upon reaching the recharge station. This value is used to verify sufficient battery capacity to reach the station and is subsequently reset to 0 at the beginning of the next slot.

Given a pair of robots performing a multi-robot task, since the task execution time is the same for both, by setting their finish slot times to be equal through (6a), we ensure that they start task execution simultaneously. In the case of a relay, (6b) equals the time from when the first robot finishes its task fragment to the time when the second robot starts executing its fragment (note that there may be a previous waiting time for synchronization). Although that would not be a proper relay, for implementation purposes, our formulation encodes as a *virtual* self-relay when a robot performs two consecutive fragments of the same task (see an example in Figure 3). Note that (6a) holds for pairs of distinct robots synchronizing, as it does not make sense for a robot to synchronize with itself in a multi-robot task. However, in (6b) a robot could relay itself, but in that case the time slots must be different. Furthermore, for each time coordination, we need to ensure that the two slots being coordinated have the same task associated with them:

$$y_{t,r_1,s_1,r_2,s_2} \leq x_{r_1,t,s_1}, \quad (7a)$$

$$y_{t,r_1,s_1,r_2,s_2} \leq x_{r_2,t,s_2}, \quad (7b)$$

$$z_{t,r_1,s_1,r_2,s_2} \leq x_{r_1,t,s_1}, \quad (7c)$$

$$z_{t,r_1,s_1,r_2,s_2} \leq x_{r_2,t,s_2}, \quad (7d)$$

$$\forall r_1, r_2 \in \mathcal{R}, s_1, s_2 \in \mathcal{S}, t \in \mathcal{T}.$$

Additionally, there are constraints on the flow of time coordination variables. First, each task fragment cannot be relayed by more than one subsequent fragment and similarly, it cannot be relaying more than one previous fragment:

$$\sum_{r_2 \in \mathcal{R}} \sum_{s_2 \in \mathcal{S}} z_{t,r_1,s_1,r_2,s_2} \leq 1, \forall r_1 \in \mathcal{R}, s_1 \in \mathcal{S}, t \in \mathcal{T}; \quad (8a)$$

$$\sum_{r_1 \in \mathcal{R}} \sum_{s_1 \in \mathcal{S}} z_{t,r_1,s_1,r_2,s_2} \leq 1, \forall r_2 \in \mathcal{R}, s_2 \in \mathcal{S}, t \in \mathcal{T}. \quad (8b)$$

Note that the above limitation does not apply to variables of type y , since a task instance in a multi-robot task must be synchronized with all the instances corresponding to the other $n_t^r - 1$ robots performing the task in parallel. Thus, (9) ensures that all instances of a given multi-robot task t (which may also be decomposable) are grouped into sets of exactly n_t^r fragments, and time synchronization only occurs between fragments belonging to the same group.

$$y_{t,r_1,s_1,r_2,s_2} = y_{t,r_2,s_2,r_1,s_1}, \quad (9a)$$

$$\forall r_1, r_2 \in \mathcal{R}, s_1, s_2 \in \mathcal{S}, t \in \mathcal{T},$$

$$(n_t^r - 1) \cdot x_{r_1,t,s_1} = \sum_{r_2 \in \mathcal{R}} \sum_{s_2 \in \mathcal{S}} y_{t,r_1,s_1,r_2,s_2}, \quad (9b)$$

$$\forall r_1 \in \mathcal{R}, s_1 \in \mathcal{S}, t \in \mathcal{T}.$$

Lastly, the total number of relays for a given task is also bounded:

$$n_t - n_t^r = \sum_{r_1 \in \mathcal{R}} \sum_{s_1 \in \mathcal{S}} \sum_{r_2 \in \mathcal{R}} \sum_{s_2 \in \mathcal{S}} z_{t,r_1,s_1,r_2,s_2}, \forall t \in \mathcal{T}. \quad (10)$$

Figure 3 depicts an example with its corresponding time coordination variables to illustrate the flow constraints in (9) and (10). It can be seen that all the task instances ($n_t = 12$) are grouped into 3 sets of 4 fragments and, within each group,

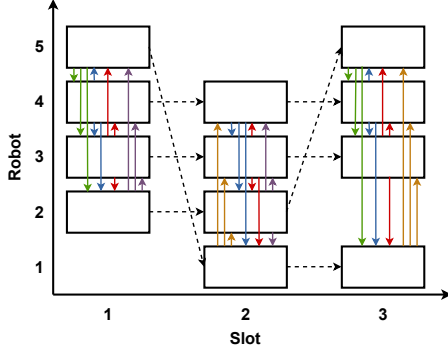


Figure 3: Time coordination example with 5 robots performing a multi-robot relayable task, which is divided into 3 fragments ($n_t^f = 3$) executed by coalitions of 4 robots ($n_t^r = 4$). Recharge tasks are not shown. Dashed black arrows indicate that the corresponding relay variable (z) is activated, and solid colored arrows indicate the activated synchronization variables (y), with a different color for each robot. A robot executing two consecutive fragments of the same task is modeled as a *self* relay.

each robot synchronizes with the other 3 performing the task. Moreover, all synchronization variables y are bidirectional and the total number of activated relay variables z in the example is $n_t - n_t^r = 8$.

E. Objective function

We propose a multi-objective cost function (11) with four terms to be minimized:⁶ 1) makespan, the time by which the last robot finishes its last task; 2) delays for task completion with respect to their deadlines; 3) waiting times for synchronization; and 4) deviation of the coalition size in multi-robot tasks with respect to the ideal size.

$$f_1 = \frac{Z}{\eta_1}, \quad (11a)$$

$$f_2 = \frac{\sum_{r \in \mathcal{R}, s \in \mathcal{S}} \Delta T_{r,s}^{max}}{\eta_2}, \quad (11b)$$

$$f_3 = \frac{\sum_{r \in \mathcal{R}, s \in \mathcal{S}} T_{r,s}^w}{\eta_3}, \quad (11c)$$

$$f_4 = \frac{\sum_{t \in \mathcal{T}} V_t}{\eta_4}, \quad (11d)$$

where η_1, η_2, η_3 , and η_4 are normalization constants so that all cost terms are on the same scale. (11a) introduces an auxiliary variable Z to encode the makespan, which can be done by adding the constraint

$$Z \geq T_{r,|S|}^f, \quad \forall r \in \mathcal{R}, \quad (12)$$

which forces Z to be greater than the completion time for each robot's queue (4c). η_1 is computed by considering a worst-case scenario in which a single robot executes all tasks, recharging when needed. In (11b), $\Delta T_{r,s}^{max}$ are slack variables

that represent the delay of robot r in completing the task assigned to slot s . We set:

$$\begin{aligned} \Delta T_{r,s}^{max} &\geq \sum_{t \in \mathcal{T}} x_{r,t,s} \cdot (T_{r,s}^f - T_t^{max}), \\ \Delta T_{r,s}^{max} &\geq 0, \quad \forall r \in \mathcal{R}, s \in \mathcal{S}. \end{aligned} \quad (13)$$

Note that since it does not make sense to consider deadlines for recharges or unassigned slots, (13) reduces to $\Delta T_{r,s}^{max} \geq 0$ when the task assigned to slot s does not belong to $\mathcal{T}$. η_2 is computed as the maximum deadline T_t^{max} , $\forall t \in \mathcal{T}$. (11c) is the normalized overall waiting time, where η_3 is calculated as the maximum battery time autonomy $\max \{B_r^{max}\}$, $\forall r \in \mathcal{R}$. Lastly, recall that our multi-robot tasks may accept *variable* coalition sizes, so (11d) penalizes tasks allocated a number of robots that is lower than their ideal coalition size N_t . This deviation in the coalition size is defined as:

$$\begin{aligned} V_t &= N_t - n_t^r, \\ V_t &\geq 0, \\ \forall t \in \mathcal{T}, N_t &> 0. \end{aligned} \quad (14)$$

For tasks with an *unspecified* coalition size, this penalty does not apply; by convention, we set $N_t = 0$ for these tasks, which is why (14) only applies when this parameter is greater than zero. For tasks with a *fixed* coalition size, an extra constraint is added to force V_t to be zero, making n_t^r and N_t coincide. η_4 is computed by adding the maximum deviation for all tasks, where this maximum deviation is $N_t - 1$; i.e., allocating a single robot to the task.

F. Optimization problem

To sum up, our complete MILP problem can be formulated as follows:

$$\begin{aligned} &\text{minimize} \quad f_1 + f_2 + f_3 + f_4 \\ &\quad \{x_{r,t,s}, T_{r,s}^w, n_t^f\} \\ &\text{subject to} \\ &\quad \text{task-slot assignment (1),} \\ &\quad \text{hardware compatibility (2),} \\ &\quad \text{counting variables (3),} \\ &\quad \text{time slot variables (4),} \\ &\quad \text{battery autonomy (5),} \\ &\quad \text{time coordination (6)–(10),} \\ &\quad \text{makespan (12),} \\ &\quad \text{task delays (13),} \\ &\quad \text{coalition size deviations (14).} \end{aligned} \quad (15)$$

G. Linearization

Finally, it is important to remark that some of the equations contain non-linear elements in the form of decision variables that are multiplied together. In particular, these non-linearities appear in (4a), (4b), (5a), (6), (9b), and (13). In order to keep the formulation as an MILP, we circumvent this issue with linearization techniques by using additional auxiliary variables [52], [53]. Basically, non-linear terms are replaced by new variables and additional linear constraints that force

⁶Note that our formulation could easily accommodate other typical objectives such as minimizing the total traveled time by all robots.

the value of the new variable to be equal to the term that it substitutes.

Thus, each product of two binary decision variables $b_1 \cdot b_2$ is replaced by a new variable $b' \in \{0, 1\}$, and the following constraints are included to force b' to be equal to the original product:

$$b' \leq b_1, \quad b' \leq b_2, \quad b' \geq b_1 + b_2 - 1. \quad (16)$$

In the same way, each product of a binary and a real⁷ decision variable $b \cdot r$, where $b \in \{0, 1\}$ and $r \in \mathbb{R}$, is replaced by a new variable $r' \in \mathbb{R}$, and the following constraints are included:

$$\begin{aligned} b \cdot r^{\min} &\leq r', & r - r^{\max} \cdot (1 - b) &\leq r', \\ b \cdot r^{\max} &\geq r', & r - r^{\min} \cdot (1 - b) &\geq r', \end{aligned} \quad (17)$$

where $r^{\min}$ and $r^{\max}$ are the lower and upper limits, respectively, of the real variable r . For the sake of brevity, the complete set of equations after linearization is not shown here, but can be accessed in the online version of our code (see Section VII).

V. HEURISTIC PLANNER

The problem posed is NP-hard, as proved in Section III-A. Therefore, solving an optimal formulation such as that presented in Section IV becomes computationally intractable as the number of robots and tasks involved increases. In this section, we propose a heuristic solver to find approximate solutions in such a way that 1) the plans comply with all problem constraints as formulated in Section IV; and 2) they can be computed efficiently enough to operate in real time.

There are problem-specific heuristics for MRTA scenarios similar to ours, in which they build an initial valid solution and then iterate over it, applying operations to improve it; e.g., merging independent tours into a single tour to save costs [2], or creating new solutions by removing random robots or tasks and rearranging them [54]. Given the complexity of our constraints, such a strategy would be hard to follow in our problem, as even minor operations may yield solutions that are not valid anymore: our time coordination constraints cannot always be fixed by adding recharges and/or adjusting waiting times. Another approach is to use metaheuristic algorithms such as genetic [37], simulated annealing [40], or LKH, which is a widely used heuristic to solve variants of TSP [27], [31]. The issue is that those approaches work more poorly in complex problems with large search space, struggling to find good solutions in reasonable time. Therefore, we propose a new heuristic algorithm leveraging properties of the problem, in which tasks are ordered following certain criteria and then assigned to robot coalitions.

Algorithm 1 summarizes our heuristic solver, which receives the set of robots $\mathcal{R}$ and tasks $\mathcal{T}$ and returns a list Q with the queue of tasks assigned to each robot. First, the algorithm decides the coalition size for each task (n_t^f) and the number of fragments into which it will be divided (n_t^e). This is done calling procedure `EstimateFragments` (Algorithm 3), which will be explained later. Then, after

Algorithm 1 HEURISTICPLANNER ($\mathcal{R}, \mathcal{T}$)

```

1:  $\{n_t^e, n_t^f, f_t\}_{t \in \mathcal{T}} \leftarrow \text{EstimateFragments}(\mathcal{R}, \mathcal{T})$ 
2: for  $r$  in  $\mathcal{R}$  do  $Q[r] \leftarrow \emptyset$  ▷ Initialize task queues
3:  $\bar{\mathcal{T}} \leftarrow \emptyset$ 
4: for  $t$  in  $\mathcal{T}$  do
5:   if  $t$  is non-decomposable or relayable then
6:      $\bar{\mathcal{T}}.add(t)$ 
7:   else
8:      $\bar{\mathcal{T}}.add(\text{Repelem}(t, n_t^f))$ 
9: while  $\bar{\mathcal{T}} \neq \emptyset$  do
10:   $Z \leftarrow \text{ComputeMakespan}()$ 
11:  for  $t$  in  $\bar{\mathcal{T}}$  do ▷ Best coalition for each task
12:     $\{\mathcal{R}_t, M_R, \Delta Z, \Delta T^w\} \leftarrow \text{SelRobots}(\mathcal{R}, t, Z)$ 
13:     $t \leftarrow \text{Sort}(\bar{\mathcal{T}}).pop()$  ▷ Allocate priority task
14:    for  $r$  in  $\mathcal{R}_t$  do
15:       $Q[r].addTaskToQueue(t, M_R[r, t], T_t^f)$ 
16: return  $Q$ 

```

initializing the robot queues, a list $\bar{\mathcal{T}}$ with all task fragments to be allocated is created (lines 3–8). For each fragmentable task t in $\mathcal{T}$, n_t^f equal fragments are repeated and added to $\bar{\mathcal{T}}$ (procedure `Repelem`), as each fragment will be allocated to a robot coalition independently. Non-decomposable and relayable tasks are included as a single element, as they will be allocated to a coalition as a whole. In each iteration of the main loop (lines 9–15), an element of $\bar{\mathcal{T}}$ is allocated until the list is empty. Given all task assignments so far, the current makespan Z of the plan is computed, and for each remaining task t in $\bar{\mathcal{T}}$, the best coalition is determined calling procedure `SelRobots` (lines 11–12). This procedure will be detailed later (Algorithm 2) and it returns the set of robots $\mathcal{R}_t$ in the best coalition selected for the task, a matrix M_R with binary flags indicating whether the robots need or do not need to include a pre-recharge to execute the task, and the increase in the makespan ΔZ and in the waiting time ΔT^w introduced when assigning the selected coalition to the task. Once the best coalitions are computed for all remaining tasks, these tasks are sorted and the one with top priority is extracted (line 13). Tasks are ordered lexicographically according to several criteria: 1) tasks that are close to their deadline $T_t^{\max}$, giving priority to those that if not assigned now, would exceed their deadline; 2) tasks whose selected coalition introduces less ΔZ ; 3) tasks whose selected coalition introduces less ΔT^w ; 4) tasks with higher execution time (T_t^e/n_t^f); 5) tasks with a higher n_t^e/n_t^c proportion; i.e., their coalition size with respect to the number of compatible robots for the task (n_t^c); and 6) tasks with less displacement time (T_t^d). The task with top priority is allocated to its selected coalition using procedure `addTaskToQueue` (lines 14–15), which adds task t to the queue of robot r , including a pre-recharge task if $M_R[r, t] == 1$ and using T_t^f (calculated in `SelRobots`) as coordination time to compute the waiting time. These task allocation heuristics are based on the optimization problem formulated in Equation 15: the task assignment order (line 13) follows similar optimization criteria, and robot coalition selection aims to minimize mission makespan. Note that the objective function in Equation 15 cannot be directly used for task ordering, as it requires adaptation for single-task scoring.

Algorithm 2 receives the set of robots $\mathcal{R}$ and the makespan

⁷The same linearization is used in the case of integer variables.

Algorithm 2 SELROBOTS($\mathcal{R}, t, Z$)

```

1:  $\mathcal{R}^c \leftarrow \text{GetAvailableRobots}()$ 
2: for  $r$  in  $\mathcal{R}^c$  do
3:    $M_R[r, t] \leftarrow 1$  if a pre-recharge is needed, 0 otherwise
4:    $T_r^f \leftarrow$  robot's finish time after  $M_R[r, t]$  and  $t$ 
5:  $\mathcal{R}_t \leftarrow \text{Sort}(\mathcal{R}^c).get(n_t^r)$   $\triangleright$  Pick earliest  $n_t^r$  robots
6:  $T_t^f \leftarrow \max_{r \in \mathcal{R}_t} T_r^f$   $\triangleright$  Task coordination time
7:  $changed \leftarrow \text{True}$ 
8: while  $changed$  do
9:    $changed \leftarrow \text{False}$ 
10:  for  $r$  in  $\mathcal{R}_t$  do
11:     $T_r^w \leftarrow$  waiting time for robot  $r$  according to  $T_t^f$ 
12:     $M_R[r, t] \leftarrow$  pre-recharge flag including  $T_r^w$ 
13:     $T_r^f \leftarrow$  robot's finish time considering  $M_R[r, t]$ 
14:     $\tilde{\mathcal{R}}_t \leftarrow \text{Sort}(\mathcal{R}^c).get(n_t^r)$   $\triangleright$  Update robot selection
15:    if  $\tilde{\mathcal{R}}_t \neq \mathcal{R}_t$  then
16:       $\mathcal{R}_t \leftarrow \tilde{\mathcal{R}}_t$ ,  $T_t^f \leftarrow \max_{r \in \mathcal{R}_t} T_r^f$ 
17:       $changed \leftarrow \text{True}$ 
18: if  $T_t^f > Z$  then  $\Delta Z \leftarrow T_t^f - Z$ 
19: else  $\Delta Z \leftarrow 0$ 
20:  $\Delta T^w \leftarrow \sum_{r \in \mathcal{R}_t} T_r^w$ 
21: return  $\mathcal{R}_t, M_R, \Delta Z, \Delta T^w$ 

```

Z produced by the current robots' task queues, and selects the best robot coalition for task t . The algorithm selects the compatible robots that finish their task queue earliest, ensuring that the constraints in problem 15 are satisfied after each task allocation. First, $\mathcal{R}^c$ is the set of robots with compatible hardware⁸ and enough maximum battery B_r^{max} to execute the task and go back to a recharging station (line 1). Second, for each compatible robot, it is determined whether a pre-recharge task has to be included or not and, according to this, the finish time of the robot queue T_r^f after including task t is calculated (lines 2–4). Third, the compatible robots are sorted according to their T_r^f and the coalition $\mathcal{R}_t$ is built selecting the earliest n_t^r robots (line 5). In order to execute the task in a coordinated manner, the coordination time T_t^f is computed; i.e., the time at which the latest selected robot will finish the task (line 6). The following procedure is then repeated iteratively (lines 8–17) until the selected coalition does not change, ensuring time coordination and battery constraints: add the required waiting time to coordinate the selected robots, recompute if any pre-recharge is needed, reorder the compatible robots with the new finish queue times, and select the earliest n_t^r robots. Convergence is guaranteed because the set of compatible robots is finite. Once a pre-recharge flag is set for $M_R[r, t]$, it is not reset. In each iteration, either a pre-recharge is added for a new robot, or the coalition remains unchanged. In the worst case, the loop terminates after all compatible robots have a pre-recharge. Lastly, once the final coalition for task t is selected, the increase in the makespan and the waiting time are computed, assuming that task assignment (lines 18–20).

Algorithm 2 has a variation for the special case of relayable tasks that are fragmented. In that case, all the fragments belonging to that task are allocated as a whole, taking into account all the available compatible robots, instead of just picking the first n_t^r (lines 5 and 14). All compatible robots are eligible for assignment to task fragments following the

specific pattern in Figure 4. This pattern satisfies battery and time coordination constraints for relayable tasks, ensuring sufficient robots are always available to perform relays while others recharge. The pattern is built with two key parameters computed in Algorithm 3; the number of fragments into which the task is divided n_t^f , and the pattern frequency f_t , which indicates the number of fragments of task t to be assigned to the same robot before introducing a recharge operation.

Algorithm 3 ESTIMATEFRAGMENTS($\mathcal{R}, \mathcal{T}$)

```

1: for  $t \in \mathcal{T}$  do
2:   if  $t$  is fixed then  $\triangleright$  Decide robots per task
3:      $n_t^r = N_t$ 
4:   else if  $t$  is variable or unspecified then
5:      $n_t^r = 1$ 
6:    $\mathcal{R}^c \leftarrow \text{GetCompatibleRobots}(t)$ 
7:    $\bar{B}_t \leftarrow \min_{r \in \mathcal{R}^c, t_1 \in \bar{\mathcal{T}}} (B_r^{max} - B_r^{min} - 2 \cdot \max T_{r, t_1, t}^d)$ 
8:   if  $T_t^e \leq \bar{B}_t$  or  $t$  is non-decomposable then
9:      $n_t^f \leftarrow 1$ ,  $f_t \leftarrow 0$ 
10:  else if  $t$  is fragmentable then
11:     $n_t^f \leftarrow \lceil T_t^e / \bar{B}_t \rceil$ ,  $f_t \leftarrow 0$ 
12:  else  $\triangleright$  Relayable tasks
13:     $n_t^c = |\mathcal{R}^c|$ ,  $n_t^e = n_t^c - n_t^r$ 
14:     $cf \leftarrow \lceil n_t^r / n_t^e \rceil$ 
15:     $n_t^f \leftarrow \lceil cf \cdot T_t^e / \bar{B}_t \rceil$ 
16:     $f_t \leftarrow \lfloor \bar{B}_t / (T_t^e / n_t^f) \rfloor$ 
17: return  $\{n_t^r, n_t^f, f_t\}_{t \in \mathcal{T}}$ 

```

Algorithm 3 first determines the coalition size for each task (lines 1–5); for simplicity, tasks with unspecified and variable coalition size are allocated to a single robot. Then, to estimate the number of fragments, it calculates the remaining battery capacity $\bar{B}_t$ for the compatible robot with the lowest capacity (lines 6–7), after accounting for the minimum safety margin and the maximum round-trip travel time to task t . Non-decomposable tasks, or tasks whose execution time is shorter than $\bar{B}_t$, are not fragmented (lines 8–9). Fragmentable tasks are split into fragments with duration shorter than $\bar{B}_t$ (lines 10–11), without the need to follow any specific pattern for recharges ($f_t = 0$). For relayable tasks, $\bar{B}_t$ and the number of excess compatible robots n_t^e are taken into account to compute n_t^f and f_t (lines 12–16)⁹. Depending on the number of excess robots available for relays, each robot may execute several consecutive fragments according to the allocation pattern in Figure 4. First, f_t consecutive fragments of the task are concatenated, followed by the corresponding recharge task, until a row with n_t^f slots is built. The same procedure is followed to create n_t^c rows, but shifting the pattern of each row one slot to the left (Figure 4a). At this point, there may be columns with more than n_t^r task fragments. For those columns, the extra fragments are replaced by recharge tasks. To reduce robot displacements and save slots, the replaced fragments are chosen from those that are preceded or followed (in their rows) by a recharge task (Figure 4b). After that, recharge tasks that are at the beginning or the end of a row can be removed and consecutive recharges merged in a single slot without modifying the relative positions of the remaining fragments (Figure 4c). Although the pattern is originally built using all

⁸For the sake of simplicity, we assume that there are enough robots to execute the available tasks, otherwise the algorithm would return that there is no valid solution.

⁹We assume sufficient robots to execute all tasks. If no excess robots were available ($n_t^e = 0$), all compatible robots would have to execute the task in parallel, precluding relays.

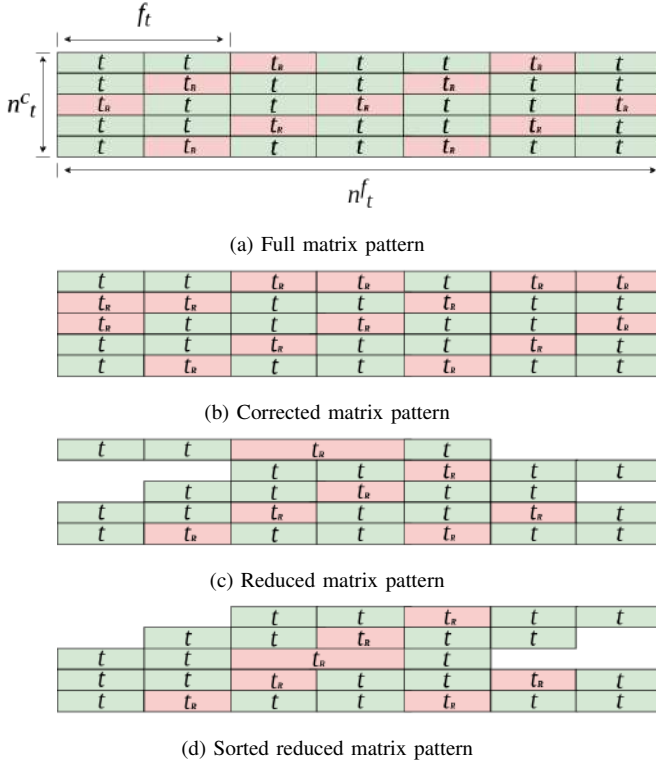


Figure 4: From top to bottom, the steps to build the robot allocation pattern for a relayable task. Example with $f_t = 2$, $n_t^f = 7$, $n_r^c = 5$, and $n_t^r = 3$.

compatible robots n_t^c , after the above steps, there may be empty rows (i.e., unused compatible robots) that would be removed. Lastly, the remaining rows are reordered according to their starting point (Figure 4d).

In summary, the robot selection in Algorithm 2 differs for relayable tasks as follows. In line 5, instead of picking the first n_r^r robots, the matrix pattern described above is built and a compatible robot is selected for each row. This is done by sorting robots according to their finish time (T_r^f) and matrix rows according to their initial time, so that robots with later finish time match rows that start later. After this, within the loop, the waiting times for each selected robot and the pre-recharge flags (in case the robot needs a recharge before starting its row) are recomputed until there are no more changes in the final T_t^f , but without varying the set of selected robots. More specifically, line 14 becomes $\tilde{T}_t^f \leftarrow \max_{r \in \mathcal{R}_t} T_r^f$, the condition of line 15 would be $\tilde{T}_t^f \neq T_t^f$, and line 16 would just be $T_t^f \leftarrow \tilde{T}_t^f$. Finally, note that in line 15 of Algorithm 1, for the case of a relayable task, procedure `addTaskToQueue` will add to each of the selected robots' queues the corresponding row of the matrix pattern as a whole, including a pre-recharge operation at the beginning if indicated by M_R .

VI. MISSION REPLANNING AND EXECUTION

In this work, we deal with dynamic scenarios where during the execution of the mission, the planning conditions may deviate from their initial states. For instance, a robot may become delayed or simply fail while performing its tasks, or

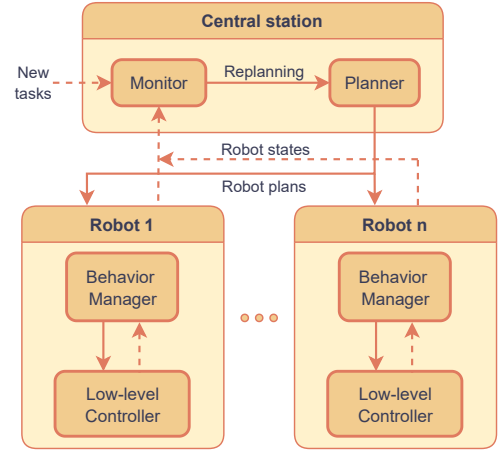


Figure 5: System architecture for mission (re-)planning and execution. The components related to task planning (top layer) run on a central station, whereas those in charge of task execution (bottom layer) are allocated on board each robot. Dashed lines depict feedback information and solid lines indicate action requests.

new task requests could arrive during the mission. In these cases, the running plan may not be valid anymore, making a replanning procedure necessary to adapt to the new circumstances. We integrate our MRTA algorithms into a mission planning and execution framework that is robust to dynamic settings, allowing online replanning due to unexpected events. The system architecture, depicted in Figure 5, was presented in our previous work [1]. The components are separated into two interleaved layers; one for mission (re-)planning and another for mission execution.

Mission planning (top layer in Figure 5) is run in a centralized manner; given the current state of the scenario (i.e., the pending tasks and the location and battery status of the available robots), the *Planner* computes an optimal plan for the team (Section V), deciding on the best task allocation. After this, each robot receives its plan, which consists of a schedule with its assigned tasks, and mission execution (bottom layer in Figure 5) is run in a distributed fashion. Each robot runs its own *Behavior Manager* onboard, which is an executive component that, for each assigned task, extracts the task parameters and activates the *Low-level Controller* to handle execution from a robot control point of view. More specifically, each Behavior Manager implements a state machine encoded as a Behavior Tree (BT) [55], which monitors task outcomes (whether execution has finished successfully or not) and the robot state (whether it has failed or become delayed). If a robot failure is detected, the BT activates contingency actions (e.g., an emergency landing in the case of a UAV running out of battery) and reports the robot's unavailability to the Monitor.¹⁰ In the absence of failures, the Behavior Manager activates the Low-Level Controller, which takes care of navigational actions and collision avoidance so that the robot can

¹⁰In our implementation, failures are a robot running out of battery or losing connectivity due to a communication dropout, but other hardware issues could easily be accommodated.

execute its task. Note that in multi-robot tasks, the Low-Level Controllers involved may need to share additional information for robot coordination. Depending on the task being executed, different behaviors will be implemented in the Low-Level Controller. For instance, an inspection task may require the robot to activate a camera and navigate through a series of waypoints, while a delivery task may involve pick-up and place actions. More details about the implementation of Low-Level Controllers and BTs for a multi-UAV inspection application can be consulted in our previous work [1].

The multi-robot plan execution is centrally supervised by the *Monitor* component at all times. This component receives feedback from the Behavior Managers indicating task/robot failures or delays. The Monitor also receives an input signal with new incoming tasks in the scenario.¹¹ In case of a robot failure, if the running plan remains valid (i.e., satisfies all problem constraints in Section IV), no action is taken. This can occur if the robot has no assigned tasks or belongs to a robot coalition without a *fixed* size (soft constraint). In the case of a robot delay, the remaining plan may no longer comply with the problem constraints regarding battery life or time coordination, and a repair operation is attempted to adapt robot schedules and maintain all constraints. Thus, a new whole plan is only required in the following circumstances: 1) the arrival of new tasks; 2) a robot failure leading to an invalid plan; or 3) an unsuccessful plan repair operation. The Monitor carries out this replanning for all pending tasks and available robots using the Planner module (Section V). Tasks already in progress are not interrupted and reallocated unless their coalition size becomes insufficient for completion.

A. Plan repair

During plan execution, every time a robot finishes a task, we check whether it is delayed; that is, whether the robot will reach the starting position of its next allocated task at a different time instant than originally planned. This could happen because the robot took longer (or less time) than expected while performing the task, or because it ended up in a different position than expected, with the consequent variation in its arrival time to its next task. However, sometimes the whole plan could be repaired by *delaying* the remaining tasks and still be a valid plan. The core idea is the following: the robot could accommodate its delay through tasks with associated waiting time by reducing their waiting times accordingly. Thus, for each future task, we accommodate part of the robot delay by updating its waiting time and delaying the corresponding start/finish task time instants accordingly. Then we propagate the remaining delay forward throughout the rest of the plan (see Algorithm 4). Any time this delay propagation reaches a time coordination point; i.e., a multi-robot task with several robots starting synchronously or a task where a relay is carried out, all the robots involved also need to update their schedules to comply with time coordination constraints (see Section IV-D). Algorithms 5 and 6 are in charge of updating robots' plans to resolve the two previous time coordination

cases. After a time coordination point is resolved, the delay by one of the robots involved may affect others' plans, and then those robots will also need to propagate forward their new delayed plans. The whole repair procedure is carried out by Algorithm 7, which sweeps the timeline of the multi-robot plan propagating delayed robot schedules, resolving coordination points as they appear. The primary objective is to restore plan validity without task reassignment, and this is done by minimizing schedule extensions (i.e., optimizing makespan).

Algorithm 4 UPDATETIMEVARS ($r, s_f, \bar{s}, \Delta t, \mathcal{R}_d$)

```

1: if  $\bar{s}[r] \geq s_f$  then return
2:  $s \leftarrow \bar{s}[r]$ ,  $\delta \leftarrow \Delta t[r]$ 
3: while  $s < s_f$  and  $\delta > 0$  do
4:    $s \leftarrow s + 1$ 
5:   if  $T_{r,s}^w > 0$  then
6:     if  $T_{r,s}^w > \delta$  then
7:        $T_{r,s}^w \leftarrow T_{r,s}^w - \delta$ 
8:        $\delta \leftarrow 0$ 
9:     else
10:       $\delta \leftarrow \delta - T_{r,s}^w$ 
11:       $T_{r,s}^w \leftarrow 0$ 
12:       $T_{r,s}^f \leftarrow T_{r,s}^f + \delta$ 
13:  $\bar{s}[r] \leftarrow s_f$ ,  $\Delta t[r] \leftarrow \delta$ 
14: if  $\Delta t[r] == 0$  then  $\mathcal{R}_d.remove(r)$ 

```

Algorithm 4 receives as input a robot r and updates its schedule, propagating its delay up to a given slot s_f . The algorithm also receives three global variables that may have to be updated at any time: $\bar{s}$ is a vector that indicates the latest slot updated so far for each robot's schedule, Δt is a vector with the current delay for each robot's schedule, and $\mathcal{R}_d$ is a list that contains all the robots with delayed plans at each moment. If the robot plan still has slots to be updated (line 1), the algorithm iterates over those slots until the target slot s_f is reached or there is no remaining delay to consume (lines 3–12). If a slot has waiting time assigned, this is reduced to accommodate the remaining delay either completely (lines 6–8) or partially (lines 9–11). The final time for the task allocated to the slot is correspondingly delayed (line 12). Before finishing, the latest slot processed for the robot schedule and its current delay are updated (line 13). If the robot has managed to accommodate all its delay within its waiting periods, its plan is no longer delayed and it is removed from the list $\mathcal{R}_d$ (line 14).

Algorithm 5 updates the schedules of a set of robots involved in the first type of coordination point; i.e., a coalition that has to start a multi-robot task synchronously. The algorithm receives $\mathcal{A}^+$ as input, which is a list of pairs (r, s) indicating 1) the identifiers of the robots involved in the synchronization, and 2) the corresponding slots within their schedules in which the coordinated task occurs. All schedules are assumed to be updated (delay propagation) up to their previous slot $s - 1$, so only the time variables corresponding to the slots where the coordination takes place need to be updated. First, all delayed robots in the coalition are checked (lines 1–11) to accommodate their delays within their waiting period either completely (lines 4–6) or partially (lines 7–10). Then the robot with maximum remaining delay in the coalition is computed (lines 12–13). If all delays have been fully

¹¹Note that a task that is not successfully finished can be considered as a new task to be re-allocated again in future plans.

Algorithm 5 UPDATESYNCHTASK ($\mathcal{A}^+, \bar{s}, \Delta t, \mathcal{R}_d$)

```

1: for  $(r, s)$  in  $\mathcal{A}^+$  do
2:    $\delta \leftarrow \Delta t[r]$ 
3:   if  $r \in \mathcal{R}_d$  then
4:     if  $T_{r,s}^w > \delta$  then ▷ Accommodate delay fully
5:        $T_{r,s}^w \leftarrow T_{r,s}^w - \delta$ 
6:        $\delta \leftarrow 0$ 
7:     else ▷ Push forward task end
8:        $\delta \leftarrow \delta - T_{r,s}^w$ 
9:        $T_{r,s}^w \leftarrow 0$ 
10:       $T_{r,s}^f \leftarrow T_{r,s}^f + \delta$ 
11:    $\bar{s}[r] \leftarrow s, \Delta t[r] \leftarrow \delta$ 
12:    $(r_{max}, s_{max}) \leftarrow \arg \max_{(r,s) \in \mathcal{A}^+} \Delta t[r]$ 
13:    $\delta_{max} \leftarrow \Delta t[r_{max}]$ 
14:   if  $\delta_{max} == 0$  then ▷ All robots synchronized
15:     for  $(r, s)$  in  $\mathcal{A}^+$  do  $\mathcal{R}_d.remove(r)$ 
16:   else
17:     for  $(r, s)$  in  $\mathcal{A}^+$  do ▷ Synch all robots with latest
18:        $\delta \leftarrow \Delta t[r]$ 
19:        $T_{r,s}^w \leftarrow T_{r,s}^w + \delta_{max} - \delta$ 
20:        $T_{r,s}^f \leftarrow T_{r,s}^f + \delta_{max} - \delta$ 
21:        $\Delta t[r] \leftarrow \delta_{max}$ 
22:        $\mathcal{R}_d.add(r)$ 

```

accommodated, the whole coalition is already synchronized (lines 14–15). Otherwise, there are still robots with remaining delay that will push the end of the multi-robot task forward, and since all must synchronize their finish times, the robot with maximum delay is taken as reference. All the other robots increase their waiting time (and their task finish time) to match the *latest* robot (lines 16–22).

Algorithm 6 UPDATERELAYTASK ($\neg\mathcal{A}, \mathcal{A}^+, \bar{s}, \Delta t, \mathcal{R}_d$)

```

1: for  $(r, s)$  in  $\mathcal{A}^+$  do
2:    $\delta \leftarrow \Delta t[r]$ 
3:   if  $r \in \mathcal{R}_d$  then
4:     if  $T_{r,s}^w > \delta$  then ▷ Accommodate delay fully
5:        $T_{r,s}^w \leftarrow T_{r,s}^w - \delta$ 
6:        $\delta \leftarrow 0$ 
7:     else ▷ Push forward task end
8:        $\delta \leftarrow \delta - T_{r,s}^w$ 
9:        $T_{r,s}^w \leftarrow 0$ 
10:       $T_{r,s}^f \leftarrow T_{r,s}^f + \delta$ 
11:    $\bar{s}[r] \leftarrow s, \Delta t[r] \leftarrow \delta$ 
12:    $(r_{max}, s_{max}) \leftarrow \arg \max_{(r,s) \in \neg\mathcal{A}, \mathcal{A}^+} \Delta t[r]$ 
13:    $\delta_{max} \leftarrow \Delta t[r_{max}]$ 
14:   if  $\delta_{max} == 0$  then
15:     for  $(r, s)$  in  $\mathcal{A}^+$  do ▷ All robots synchronized
16:        $\mathcal{R}_d.remove(r)$ 
17:   else
18:     for  $(r, s)$  in  $\neg\mathcal{A}, \mathcal{A}^+$  do ▷ Synch all robots with latest
19:        $\delta \leftarrow \Delta t[r]$ 
20:        $T_{r,s}^w \leftarrow T_{r,s}^w + \delta_{max} - \delta$ 
21:        $T_{r,s}^f \leftarrow T_{r,s}^f + \delta_{max} - \delta$ 
22:        $\Delta t[r] \leftarrow \delta_{max}$ 
23:        $\mathcal{R}_d.add(r)$ 

```

Algorithm 6 updates the schedules of a set of robots involved in the second type of coordination point, a coalition (or single robot) that is relayed by another coalition (or single robot). This time the algorithm receives as input two lists, $\neg\mathcal{A}$ and $\mathcal{A}^+$: the former contains pairs (r, s) with the robots being relayed and the slots that are relayed in their corresponding schedules; the latter contains pairs (r, s) with the relaying robots and their relaying slots. All schedules are assumed

to be updated (delay propagation) up to the slot previous to the relay; note that this is s for robots in $\neg\mathcal{A}$ and $s - 1$ for robots in $\mathcal{A}^+$. The algorithm starts accommodating all possible delay for the robots in $\mathcal{A}^+$ within their waiting period (lines 1–11), as was done in Algorithm 5. Then the robot with maximum remaining delay from the relayed and relaying sets is computed (lines 12–13). In the case of a relay, both the relayed and relaying robots must be time coordinated, and the *latest* robot will determine the new relay time instant. If all delays become fully accommodated, the coordination point is already solved (lines 14–16). Otherwise, the relay instant is put off according to the maximum delay, and all robots involved increase their waiting time (and their task finish time) to match the *latest* robot (lines 18–23). An example of this procedure is depicted in Figure 6.

Algorithm 7 REPAIRPLANS ($t_0, r', \delta', \bar{s}$)

```

1: if  $\delta' \leq 0$  then
2:    $s' \leftarrow \bar{s}[r'], T_{r',s'+1}^w \leftarrow T_{r',s'+1}^w + |\delta'|$ 
3:   return True
4: else
5:    $\mathcal{R}_d \leftarrow \emptyset, \mathcal{R}_d.add(r')$ 
6:   for  $r$  in  $\mathcal{R}$  do
7:      $\Delta t[r] \leftarrow 0$ 
8:    $\Delta t[r'] \leftarrow \delta'$ 
9:    $\mathcal{C} \leftarrow \text{GetOrderedCoordinationTasks}(t_0)$ 
10:  for  $(\neg\mathcal{A}, \mathcal{A}^+)$  in  $\mathcal{C}$  do
11:    for  $(r, s)$  in  $\mathcal{A}^+$  do
12:      UpdateTimeVars( $r, s - 1, \bar{s}, \Delta t, \mathcal{R}_d$ )
13:    if  $\neg\mathcal{A} \neq \emptyset$  then
14:      for  $(r, s)$  in  $\neg\mathcal{A}$  do
15:        UpdateTimeVars( $r, s, \bar{s}, \Delta t, \mathcal{R}_d$ )
16:      UpdateRelayTask( $\neg\mathcal{A}, \mathcal{A}^+, \bar{s}, \Delta t, \mathcal{R}_d$ )
17:    else
18:      UpdateSynchTask( $\mathcal{A}^+, \bar{s}, \Delta t, \mathcal{R}_d$ )
19:    if  $\mathcal{R}_d == \emptyset$  then break
20:  for  $r$  in  $\mathcal{R}_d$  do
21:    UpdateTimeVars( $r, |S|, \bar{s}, \Delta t, \mathcal{R}_d$ )
22:   $valid \leftarrow \text{True}$ 
23:  for  $r$  in  $\mathcal{R}$  do
24:     $valid \leftarrow valid \cap \text{checkPlanBattery}(r)$ 
25:  return valid

```

Algorithm 7 attempts to repair a multi-robot plan given that one of the robots r' has finished its assigned task at t_0 and has a delay δ' before its next scheduled task. The algorithm receives as input the vector $\bar{s}$, which indicates the current running slot for each robot's schedule. If the delay is negative, the robot is ahead of its schedule and a waiting period for its next slot is added for synchronization (lines 1–4). Otherwise, after initializing variables (lines 5–8), a time ordered list $\mathcal{C}$ with all the coordination points in the multi-robot schedule starting at t_0 is computed (line 9). Each item in $\mathcal{C}$ is a pair $(\neg\mathcal{A}, \mathcal{A}^+)$ with all the information on the robots involved in the corresponding coordination point. The algorithm synchronizes robots' schedules for all coordination points until there are no more delayed robots (lines 10–19). Depending on the type of coordination point, Algorithm 5 or Algorithm 6 is used. Before each coordination point is synchronized, Algorithm 4 propagates delays in the schedules of the involved robots up to the slot preceding the coordination point. Given the definitions of $\mathcal{A}^+$ and $\neg\mathcal{A}$, this will be up to $s_f = s - 1$ or $s_f = s$, respectively. Finally, after all coordination points are swept,

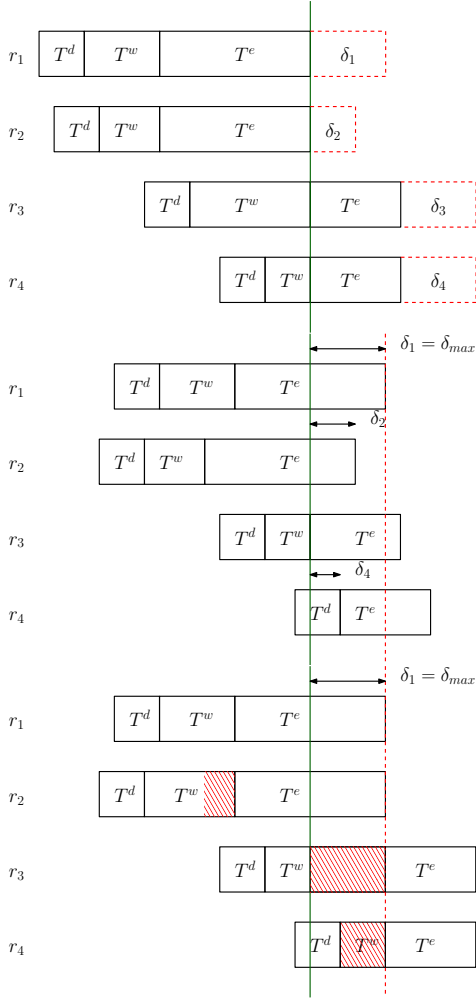


Figure 6: Example of Algorithm 6 repairing the schedules of a relay point. Top view: originally, robots r_1 and r_2 ($\neg\mathcal{A}$) execute a multi-robot task which is relayed by robots r_3 and r_4 ($\mathcal{A}^+$) at the time line in green. Dashed red squares represent the delay for each robot schedule. Middle view: first, the relays of robots r_3 and r_4 are partially accommodated by reducing waiting times; relays of robots in $\neg\mathcal{A}$ (r_1 and r_2) were already propagated before Algorithm 6. The maximum delay (r_1) determines the new relay time instant (dashed red vertical line). Bottom view: robots synchronize with the new relay instant by extending their waiting periods (red patterns).

all robots' schedules are delay propagated up to their last slot ($s_f = |S|$) and the resulting plans are checked for battery compliance (lines 20–24). If the repaired plan does not comply with battery constraints, the algorithm reports a failure.

VII. EXPERIMENTAL RESULTS

This section presents experimental results that evaluate our methods. Given the use case in Section VII-A and the experimental setup in Section VII-B, we present numerical experiments (Sections VII-C and VII-D) with a twofold objective: (i) evaluate optimal solutions and demonstrate the potential of our MILP formulation to solve more complex scenarios, due to our higher degree of freedom in terms of task



Figure 7: Photovoltaic solar plant in Evora (Portugal). A single recharging station is located in the middle (circle mark).

decomposability and coalition size flexibility; and (ii) assess our heuristic planner performance and test its scalability for larger scenarios. Furthermore, we present a realistic simulation to demonstrate the feasibility of our approach in real applications and show the advantages of our replanning framework in dynamic scenarios (Section VII-E).

A. Use case description

We define a use case taking as inspiration our running example in Section III: a team of UAVs that provides support to human workers during inspection and maintenance operations in a solar energy plant. We used an actual photovoltaic facility (see Figure 7) of size approximately 200×300 m located in Evora, Portugal. The UAVs can help with three types of tasks: 1) *inspection*, in which they capture visual or thermal images of a given remote area with solar panels; 2) *monitoring*, in which they provide the supervising team with a view of a human worker operating on the plant, in order to assess his/her safety; and 3) *delivery*, in which they transport and hand out items such as tools or parts to a human worker. According to our categorization in Section III, inspection tasks are fragmentable (the target area could be divided), monitoring tasks are relayable (a continuous videostream of the worker is required in risky operations), and delivery tasks are non-decomposable. The team is heterogeneous: depending on its payload, each UAV can conduct certain tasks. Specific cameras enable inspection and monitoring tasks, while a load transportation system enables delivery tasks. The maximum flight time for each vehicle is $B_r^{max} = 20$ minutes and its traveling speed $v_r = 5$ m/s. The recharging station can be used by multiple vehicles simultaneously and is located at the central point of the plant. Each recharge operation takes $T_{tr}^e = 5$ minutes.

B. Experimental setup

We implemented our code in MATLAB R2022b¹² using Gurobi to compute exact optimal solutions. All experiments were run with an Intel 8-core i7-7700 CPU@3.60GHz with 15.6GB RAM. The methods were evaluated over a set of random scenarios based on the use case in Section VII-A. UAVs start at random positions within the solar plant, with an initial consumed battery time $B_{r,0}$ uniformly sampled from 0,

¹²Code at https://github.com/multirobot-use/mrta_heuristic_planner.

0.25, or 0.5 of the total flight time B_r^{max} . Tasks are also placed at random positions with a deadline $T_t^{max} = 100$ minutes and an execution time T_t^e uniformly sampled from 0.35, 1.25, or 2.5 of the total flight time (this duration is set to $0.35 \cdot B_r^{max}$ for tasks that cannot be fragmented, so they do not last longer than the total flight time). Each task has a decomposability uniformly sampled from $\{non-decomposable, fragmentable, relayable\}$ and a coalition size flexibility sampled from $\{fixed, variable, unspecified\}$. The specified coalition size N_t is sampled from 1 to the maximum number of compatible UAVs. The hardware capabilities for each UAV are uniformly sampled from three different types. Given the set of sampled UAV types in the team, each task is then randomly set as compatible or incompatible (with a 0.5 probability) for each of these hardware types, ensuring compatibility with at least one type.

In order to evaluate the multi-robot plans for each method in a given set of scenarios, we defined the following metrics:

- 1) *Success Rate (SR)*: Percentage of scenarios for which a solution is found.
- 2) *Recharge Rate (RR)*: Percentage of the solved scenarios that have at least one recharge.
- 3) *Number of Recharges (NR)*: Total number of recharges in a solution.
- 4) *Objective function value (f)*: Value of the function (11) for a solution. This is a cost to be minimized.
- 5) *Makespan (Z)*: Makespan, time by which the latest robot finishes its plan.
- 6) *Waiting Time Rate (WTR)*: Percentage of time each robot is waiting out of its whole plan duration, averaged over all robots in the plan.
- 7) *Coalition Size Deviation (CSD)*: Relative coalition size deviation for multi-robot tasks V_t/N_t , averaged for all tasks in the plan with a *variable* coalition size.
- 8) *Consumed Battery Time (CBT)*: Average battery time consumed per robot in a plan, taking into account that the execution and waiting time during recharge tasks do not consume battery time.
- 9) *Workload Distribution (WD)*: Percentage of time each robot's plan lasts relative to the whole mission duration (makespan), averaged over all robots in the plan. If the work were equally distributed, all robots would finish simultaneously at the makespan, and this metric would be 100%. Therefore, the higher its value the better.
- 10) *Computation Time (CT)*: Time needed to compute a solution.

C. Small-scale scenarios

In this section, we show numerical experiments to demonstrate the potential of our MILP formulation. We prove that by including features such as task decomposition and coalition size flexibility, our approach finds feasible solutions for scenarios that would be unsolvable otherwise. We evaluate the quality of optimal solutions when certain features are removed from the formulation, in order to analyze their influence on the results. Lastly, we compare the performance of our heuristic solver (Section V) against exact optimal solutions.

We generated 100 random scenarios following the methodology explained in Section VII-B. Since the number of variables in our MILP formulation increases significantly with the size of the multi-robot team and the number of tasks in the mission, numerical solvers such as Gurobi do not scale with large scenarios. Therefore, we limited this first experiment to small-scale scenarios where Gurobi was able to find solutions without running out of memory. In particular, the scenarios created had missions with 2 tasks (plus the required recharge tasks) and 3 available UAVs. Due to the size of the scenarios, we did not limit the hardware capabilities of the robots, so all UAVs were able to execute any of the tasks. Although small, these scenarios exhibit enough complexity to yield plans with a rich variety of features such as recharges, task fragmentation/relays, and multi-robot synchronization. Therefore we believe they can be useful for drawing some conclusions about the characteristics of optimal solutions.¹³

We compared different approaches to solve the scenarios: 1) *Complete*: optimal plans for our complete MILP formulation (15); 2) *No-Fragmentation*: optimal plans for our MILP formulation considering all tasks non-decomposable; 3) *Fixed-Coalition-Size*: optimal plans for our MILP formulation considering all tasks with fixed coalition size (undefined tasks are always assigned to a single robot in this variant); 4) *Incomplete*: optimal plans for our MILP formulation combining Fixed-Coalition-Size and No-Fragmentation variants; 5) *Heuristic*: plans obtained with our heuristic solver. Figure 8 shows the results. Makespan, WTR, and CSD are explicitly minimized in our MILP formulation through the objective function f . The Complete variant outperforms the Incomplete one across all these metrics except for CSD, which is inherently zero in variants with fixed coalition sizes. The Complete variant also performs better than the Incomplete variant for metrics not explicitly optimized, namely CBT and WD. This demonstrates that leveraging task fragmentation and coalition size flexibility improves plan quality for solvable scenarios. Similar results are observed for the Complete and No-Fragmentation variants, and for the Fixed-Coalition-Size and Incomplete variants, respectively. This might suggest that fragmentation offers no advantage. However, the SR was 100% for the Complete variant, compared to 37% for No-Fragmentation, 92% for Fixed-Coalition-Size, 36% for Incomplete, and 100% for Heuristic. This indicates that fragmentation significantly increases the number of solvable scenarios. The high SR for Fixed-Coalition-Size stems from our generation of scenarios predominantly solvable with fixed coalition sizes. Note that the metrics in Figure 8 are calculated only for scenarios solvable by all approaches, meaning fragmentation was not strictly required. Nonetheless, fragmentation may still be beneficial even when not mandatory, as it allows task execution at different times, potentially improving battery utilization and reducing recharges. Such situations did not arise in these small-scale scenarios. Regarding recharges, the Complete and No-Fragmentation variants had an RR of 5.56% with an average NR of 0.08 per scenario; Fixed-Coalition-Size and Incomplete had RR of 16.67% and average NR of 0.28;

¹³Video with illustrative examples at: <https://youtu.be/a6zXwTZKFR4>

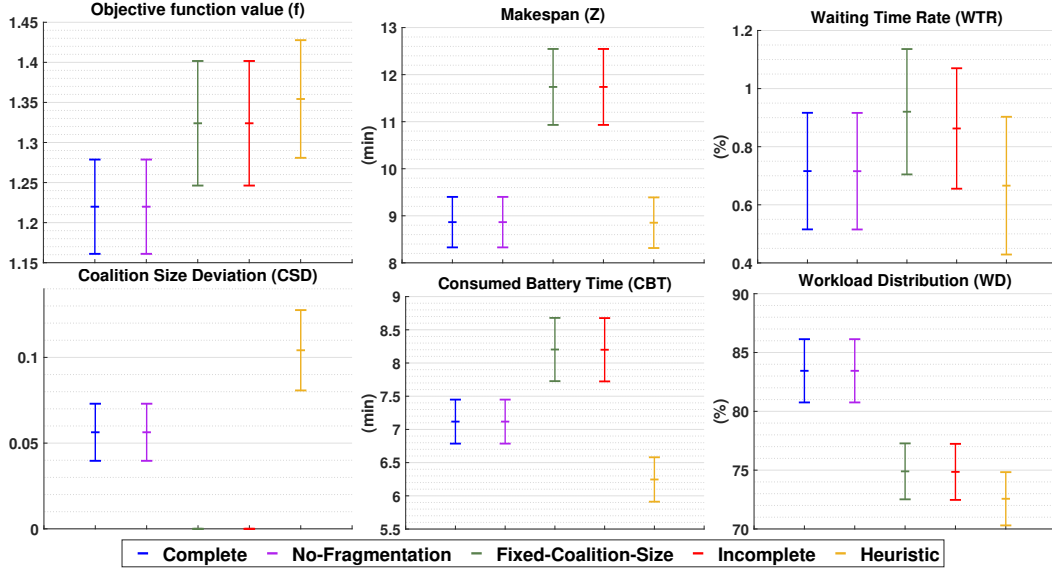


Figure 8: Resulting metrics for the small-scale scenarios. Mean and standard deviation are shown for the scenarios that are solvable with the five approaches being compared.

and Heuristic had RR of 5.56% and average NR of 0.06. This suggests that the Complete version also achieves more efficient solutions than the Incomplete version by requiring fewer recharges, and the Heuristic solver performs similarly to the Complete version in terms of recharges.

Overall, heuristic solutions show worse performance in terms of objective function value f , as expected. Nonetheless, the heuristic solver found the optimal solution (i.e., a value for f equal to the one in the Complete solution) in a 25% of the scenarios. In addition, it is important to note that the Heuristic approach performed as the Complete in makespan, which is the most critical optimization criterion, and even outperformed the others in CBT and WTR.

D. Scalability test

The average computation times for the small-scale scenarios were 1,450.656 s, 1.525 s, and 0.017 s, for the Complete, Incomplete and Heuristic approaches, respectively. This demonstrates the lack of scalability for optimally solving our complete MILP formulation, due to the large number of variables and constraints. Table II shows the number of variable and constraints, categorized by type, for a range of scenarios with increasing numbers of robots and tasks. The size of the MILP instances grows exponentially, with a consistently significant proportion of integer variables, which are known to impact scalability. Among the problem's inherent variables and constraints, those related to time coordination are particularly numerous. This aligns with the understanding that the primary complexity stems from task fragmentation and multi-robot coordination. However, the overhead associated with linearization also contributes substantially to the problem's size. In summary, the results confirm that the MILP formulation's intractability is due to the large size of the resulting instances. Contrary to ours, sequence-based formulations use binary variables to indicate whether a task i

Table II: MILP formulation size as the number of robots/tasks (n/m) increases. Values shown represent the worst case from 100 randomly generated scenarios. Scenarios below the horizontal line could not be solved with Gurobi.

Scenario size	Basic variables	Time related variables	Time coordination variables	Linearization variables (%)	Integer / Real variables (%)	Total variables
$n=1 / m=1$	34	74	77	61.08	56.76 / 43.24	185
$n=1 / m=2$	60	212	534	69.98	46.4 / 53.6	806
$n=2 / m=1$	66	292	1,040	69.24	39.41 / 60.59	1,398
$n=2 / m=2$	107	564	3,644	71.47	36.41 / 63.59	4,315
$n=2 / m=3$	176	1,156	12,208	72.08	34 / 66	13,540
$n=2 / m=5$	292	2,404	31,704	72.45	33.34 / 66.66	34,400
$n=3 / m=2$	254	1,791	36,579	71.6	31.18 / 68.82	38,624
$n=3 / m=3$	335	2,598	61,468	71.8	31.06 / 68.94	64,401
$n=2 / m=10$	771	10,144	189,896	72.53	32.17 / 67.83	200,811
$n=5 / m=2$	538	3,490	201,966	71.42	29.45 / 70.55	205,994
$n=5 / m=5$	1,824	16,670	2,102,357	71.54	29.08 / 70.92	2,120,851
$n=10 / m=2$	2,093	14,520	3,501,506	71.43	28.78 / 71.22	3,518,119
$n=10 / m=10$	29,851	479,720	423,639,104	71.45	28.65 / 71.35	424,148,675
Scenario size	Basic constraints	Time related constraints	Time coordination constraints	Linearization constraints (%)	-	Total constraints
$n=1 / m=1$	38	218	285	83.55	-	541
$n=1 / m=2$	81	662	2,030	87.41	-	2,773
$n=2 / m=1$	102	868	3,957	87.36	-	4,927
$n=2 / m=2$	173	1,764	13,970	88.41	-	15,907
$n=2 / m=3$	304	3,700	46,939	88.79	-	50,943
$n=2 / m=5$	512	7,804	122,051	88.99	-	130,367
$n=3 / m=2$	490	5,616	140,810	88.76	-	146,916
$n=3 / m=3$	637	8,322	236,749	88.86	-	245,708
$n=2 / m=10$	1,417	32,764	731,826	89.11	-	766,007
$n=5 / m=2$	1,115	11,050	778,346	88.81	-	790,511
$n=5 / m=5$	3,752	54,890	8,107,103	88.89	-	8,165,745
$n=10 / m=2$	4,545	46,020	13,503,026	88.87	-	13,553,591
$n=10 / m=10$	61,697	1,549,820	1,634,005,986	88.89	-	1,635,617,503

follows a task j for a given shared resource, typically reducing the number of variables. However, the inherent complexity of our problem would still hinder scalability. For this reason, we opted not to pursue these alternative formulations, which are also less intuitive in our case. Our significant overhead from linearization suggests that exploring Non-Linear Programming solvers could be a promising line of work. Nonetheless, this is beyond the scope of our current work. Given the problem's complexity, scalability challenges would likely persist even with more efficient exact solvers, reinforcing the need for heuristic solutions.

Next, we test the scalability of our heuristic planner further. For this, we generated a set of random scenarios as described

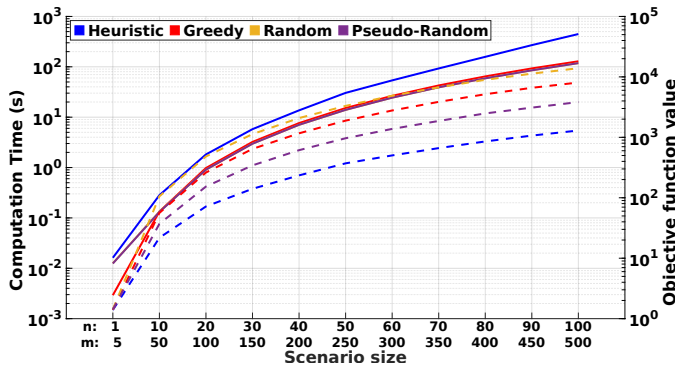


Figure 9: Evolution of computation time (solid lines) and the objective function value f (dashed lines) for heuristic variants tested as the size of the scenarios (number of robots/tasks) increases. Average values for 100 scenarios are shown. Both metrics are shown on a logarithmic scale.

in Section VII-B, gradually increasing the size of the problem in terms of number of tasks and UAVs, running 100 scenarios per problem size. We then solved each scenario with several variants of the heuristic planner: 1) *Heuristic*: our heuristic planner as described in Section V; 2) *Random*: a version of our heuristic planner where the order in which tasks are allocated and the selection of robots are decided randomly; 3) *Pseudo-Random*: a pseudo-random version where only the task allocation order is computed randomly, while robot selection is carried out using Algorithm 2; and 4) *Greedy*: a simpler greedy heuristic algorithm used as a baseline. It is inspired by the way that food buffets work. A predefined fixed order is used to build two queues: one with tasks (according to their execution time in descending order) and one with robots (in numerical order). Then the robots iteratively take as many tasks as they can handle before going to recharge and returning to the end of the queue, until all tasks are covered. Note that the four variants take care of coordination of the robot coalitions and of complying with all problem constraints, ensuring that all final solutions are valid plans.

Figure 9 shows the results of the scalability tests, both in terms of computation time and quality of the solutions. The quality of solutions is measured with the value of the objective function f ; the lower, the better. From this perspective, our heuristic algorithm outperforms the others, with the improvement over the other variants increasing exponentially as the size of the scenario grows. The Random version gives the worst results, followed by the Greedy version, which demonstrates the importance of applying a more *intelligent* heuristic for such a highly constrained problem. Moreover, although the Pseudo-Random version performs better than Random and Greedy, it still produces significantly worse solutions than our Heuristic approach, which shows the importance of task allocation order. In terms of computation time, our Heuristic approach scales worse than the other variants, which show similar performance. Recall that the Heuristic variant recomputes the best robot coalition for each task at each iteration, and this robot selection also implies iterating over several coalitions until the best one is chosen. Therefore, the

increase in computation time with the size of the scenario was expected. The other variants tested avoid some of this computation, as they simply pick tasks or robots randomly (Random and Pseudo-Random) or according to a predefined order (Greedy). Overall, although the computation time increase is exponential for our heuristic solver, the results show that we can handle very large scenarios, with a computational load that is reasonable for real-time planning (on the order of minutes in the worst case).

E. Plan repair and replanning

In this section, we evaluate our whole mission replanning and execution framework in dynamic scenarios through a realistic simulation setup. The objective is twofold: we assess the performance of our algorithms for plan repair and replanning under robot delays and failures, and we show the feasibility of our approach for real applications.

Table III: Plan repair and replanning performance. The relative increments in the metrics are averaged over the successful scenarios for each column.

Approach	Repair		Replanning		Combined	
	Short	Long	Short	Long	Short	Long
SR (%)	89.5	23	100	45.5	100	45.5
Δf (%)	3.47 $\pm$ 0.46	30.81 $\pm$ 5.02	65.38 $\pm$ 6.67	83.68 $\pm$ 8.79	11.73 $\pm$ 3.61	53.15 $\pm$ 5.70
ΔZ (%)	0.20 $\pm$ 0.02	1.77 $\pm$ 0.30	1.74 $\pm$ 0.28	4.28 $\pm$ 0.60	0.60 $\pm$ 0.16	3.65 $\pm$ 0.52
ΔWTR (%)	2.11 $\pm$ 0.39	22.53 $\pm$ 4.40	48.15 $\pm$ 5.10	56.50 $\pm$ 6.41	7.44 $\pm$ 2.07	37.51 $\pm$ 4.59
ΔCBT (%)	0.26 $\pm$ 0.02	0.31 $\pm$ 0.15	0.23 $\pm$ 0.02	0.27 $\pm$ 0.11	0.25 $\pm$ 0.02	0.26 $\pm$ 0.10
ΔWD (%)	0.02 $\pm$ 0.01	0.29 $\pm$ 0.19	5.78 $\pm$ 0.72	5.30 $\pm$ 1.09	0.86 $\pm$ 0.36	2.20 $\pm$ 0.71

In a first experiment, we assessed the effectiveness of our plan repair and replanning methods using random trials involving unexpected robot delays. We generated 200 random scenarios (as described in Section VII-B), each containing 50 tasks and 10 UAVs. For each scenario, we computed an initial plan using Algorithm 1 and then randomly selected a UAV from the plan and a task from its schedule to apply a delay. We generated 400 trials in total: the first 200 trials involved sampling short delays uniformly distributed between 30 seconds, 1 minute, and 2 minutes for the previously computed plans; the other 200 trials involved sampling long delays uniformly distributed between 10, 15, and 20 minutes. Long delays were only applied to recharge tasks, as such delays would otherwise render the UAV inoperative due to battery depletion.

Table III compares three approaches for handling delayed plans: repairing them with Algorithm 7, replanning from the current state with Algorithm 1, or our approach, replanning in case repairing fails.¹⁴ The repair approach exhibits a high success rate, which decreases with longer delays, as expected. Replanning may fail when the delayed robots are depleted of battery and insufficient robots remain to complete the mission. For the repair approach, short delays minimally affect performance metrics due to the algorithm's ability to distribute the delay across existing waiting times. The impact is more pronounced for longer delays, primarily affecting the WTR (and consequently, f), as these delays exceed the initial waiting times and necessitate additional waiting periods. CBT remains unaffected because waiting times are strategically placed during recharges whenever possible. WD is also

¹⁴Video with illustrative examples at: <https://youtu.be/a6zXwTZKFR4>

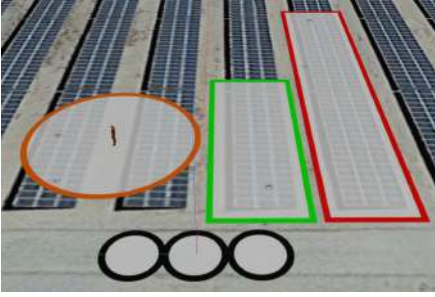


Figure 10: Simulation of the solar plant use case. Orange, green and red areas represent monitoring and inspection zones.

minimally impacted due to inter-robot synchronization; when one queue is delayed, others adjust proportionally, maintaining balanced WD. For replanning, results are similar for both short and long delays, as this approach does not explicitly accommodate existing delays. WTR (and consequently, f) is significantly worse after replanning. The heuristic planner disregards the initial plan and generates a new one from the delayed state, often leading to substantially different solutions. These results demonstrate that combining repair with replanning, rather than simply recomputing a new plan, leads to a higher SR and more effective handling of robot delays in terms of performance. Moreover, in real-world operations, repairing a plan offers practical benefits by avoiding task reassignment: once a multi-UAV flight plan is established, operators typically prefer adjusting waiting times, as reassignment requires new flight plans and complicates safety checks.

Finally, we ran an experiment to demonstrate the replanning feature through an illustrative example. For that, we implemented our system architecture for mission replanning and execution (Section VI) integrated with ROS (Robot Operating System), and we created a Gazebo simulation of the Evora solar plant for our use case (Section VII-A).¹⁵ The UAVs were modeled using a software-in-the-loop tool to simulate the PX4 autopilot firmware. This setup allowed us to reproduce real application scenarios quite closely, in which the system could not distinguish between simulated or actual UAVs. Our MATLAB code for mission planning was integrated into the ROS architecture using the ROS Toolbox for MATLAB.¹⁶ Using ROS Actionlib, we created an *Action Server* in MATLAB that receives planning requests from a High-Level Planner module in ROS and communicates the resulting plans to the UAVs. In the experiment, a team with 3 available UAVs was assigned a mission with 3 tasks: 2 *inspection* tasks, and 1 *monitoring* task. While executing the initial plan, computed by the heuristic planner, where each robot was assigned one task, we simulated a failure in the UAV performing the *monitoring* task, which was then reassigned to one of the other UAVs. Figure 10 shows a screenshot of the simulation.¹⁷

VIII. CONCLUSIONS

In this paper, we have presented a planning framework for heterogeneous MRTA in long-endurance missions for dynamic scenarios. We have formulated an optimization problem as a MILP that integrates robot recharges, heterogeneous robot capabilities, task fragmentation/relays, and time coordination for multi-robot tasks. Our results demonstrate that the aggregation of such a diverse set of features can help us solve complex missions that are relevant to a wide spectrum of multi-robot applications that would otherwise be unsolvable. To achieve better scalability for large scenarios and real-time planning performance, we have also proposed a heuristic solver for our MRTA problem and integrated it into a mission planning and execution architecture capable of repairing or recomputing plans online as unexpected circumstances arise. Our experimental results in a realistic use case for multi-UAV inspection demonstrate that 1) the flexibility introduced by our decomposable and varying coalition size tasks enables us to improve the defined performance metrics for the resulting plans; 2) our heuristic solver can scale for large scenarios in terms of computation time and outperform other similar variants in terms of efficacy; and 3) our replanning method can repair plans for unexpected robot delays.

It is important to highlight that the complexity of our MRTA problem makes it difficult to find an alternative planner for comparison in the state of the art. While we have discussed some related works in Section II, we did not find scalable solvers in the literature that can tackle problems integrating all our constraints simultaneously. Metaheuristic methods that can solve generic optimization problems, such as genetic algorithms or simulated annealing, could be an option, but the standard implementations of these methods do not exploit inherent problem properties to achieve better performance and do not scale when the number of variables in the formulation increases significantly. For example, we have tried to solve the scenarios in Section VII-D with the genetic algorithm implemented in the MATLAB Optimization Toolbox without success. An interesting direction for future work could be the design of a tailored metaheuristic algorithm that leverages particular features of the problem to improve efficiency in the search for possible solutions, and then use it for comparison. The exploration of more efficient MILP formulations is another promising avenue for future work. Finally, we plan to conduct field experiments with a real team of UAVs performing inspection missions to demonstrate our planning framework, evaluating communication latency and overhead.

REFERENCES

- [1] A. Calvo, G. Silano, and J. Capitan, "Mission Planning and Execution in Heterogeneous Teams of Aerial Robots supporting Power Line Inspection Operations," in *ICUAS*, 2022, pp. 1644–1649.
- [2] S. Agarwal and S. Akella, "Line coverage with multiple robots: Algorithms and experiments," *IEEE Trans. Robot.*, vol. 40, pp. 1664–1683, 2024.
- [3] B. A. Ferreira, T. Petrović, M. Orsag, J. R. Martínez-de Dios, and S. Bogdan, "Distributed allocation and scheduling of tasks with cross-schedule dependencies for heterogeneous multi-robot teams," *IEEE Access*, vol. 12, pp. 74 327–74 342, 2024.
- [4] K. Leahy *et al.*, "Scalable and robust algorithms for task-based coordination from high-level specifications (ScRATHeS)," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2516–2535, 2022.

¹⁵Code at https://github.com/multirobot-use/mrta_execution_architecture.

¹⁶<https://www.mathworks.com/products/ros.html>

¹⁷Full video available at: <https://youtu.be/2hzP7LZRd0g>

- [5] E. Seraj, L. Chen, and M. C. Gombolay, "A hierarchical coordination framework for joint perception-action tasks in composite robot teams," *IEEE Trans. Robot.*, vol. 38, no. 1, pp. 139–158, 2022.
- [6] M. Krizmancic, B. Arbanas, T. Petrovic, F. Petric, and S. Bogdan, "Co-operative aerial-ground multi-robot system for automated construction tasks," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 798–805, 2020.
- [7] A. J. Smith, G. Best, J. Yu, and G. A. Hollinger, "Real-time distributed non-myopic task selection for heterogeneous robotic teams," *Autonomous Robots*, vol. 43, no. 3, pp. 789–811, 2019.
- [8] A. Calvo and J. Capitan, "Optimal Task Allocation for Heterogeneous Multi-robot Teams with Battery Constraints," in *ICRA*, 2024, pp. 7243–7249.
- [9] B. P. Gerkey and M. J. Mataric, "A formal analysis and taxonomy of task allocation in multi-robot systems," *Int. J. Rob. Res.*, vol. 23, no. 9, pp. 939–954, 2004.
- [10] E. Nunes, M. Manner, H. Mitiche, and M. Gini, "A taxonomy for task allocation problems with temporal and ordering constraints," *Robot. Auton. Syst.*, vol. 90, pp. 55–70, 2017.
- [11] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *Int. J. Rob. Res.*, vol. 32, no. 12, pp. 1495–1512, 2013.
- [12] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, 2009.
- [13] M. Otte, M. J. Kuhlman, and D. Sofge, "Auctions for multi-robot task allocation in communication limited environments," *Autonomous Robots*, vol. 44, pp. 547–584, 2020.
- [14] S. Chopra, G. Notarstefano, M. Rice, and M. Egerstedt, "A distributed version of the hungarian method for multirobot assignment," *IEEE Trans. Robot.*, vol. 33, no. 4, pp. 932–947, 2017.
- [15] L. Liu, N. Michael, and D. A. Shell, "Communication constrained task allocation with optimized local task swaps," *Autonomous Robots*, vol. 39, pp. 429–444, 2015.
- [16] M. McIntire, E. Nunes, and M. Gini, "Iterated multi-robot auctions for precedence-constrained task scheduling," in *AAMAS*, 2016, pp. 1078–1086.
- [17] X. Bai *et al.*, "Group-based distributed auction algorithms for multi-robot task assignment," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1292–1303, 2023.
- [18] A. Torreño, E. Onaindia, A. Komenda, and M. Štolba, "Cooperative multi-agent planning: A survey," *ACM Computing Surveys*, vol. 50, no. 6, 2017.
- [19] N. Dhanaraj *et al.*, "Multi-robot task allocation under uncertainty via hindsight optimization," in *ICRA*, 2024, pp. 16 574–16 580.
- [20] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Autonomous Robots*, vol. 46, no. 1, pp. 231–247, 2022.
- [21] S. Shekhar and R. I. Brafman, "Representing and planning with interacting actions and privacy," *Artif. Intell.*, vol. 278, p. 103200, 2020.
- [22] S. S. Chouhan and R. Niyogi, "MAPJA: Multi-agent planning with joint actions," *Applied Intelligence*, vol. 47, no. 4, pp. 1044–1058, 2017.
- [23] A. Corberan *et al.*, "Arc routing problems: A review of the past, present, and future," *Networks*, vol. 77, no. 1, pp. 88–115, 2021.
- [24] T. Vidal, G. Laporte, and P. Matl, "A concise guide to existing and emerging vehicle routing problem variants," *Eur. J. Oper. Res.*, vol. 286, no. 2, pp. 401–416, 2020.
- [25] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 47, no. 1, pp. 70–85, 2017.
- [26] M. Li *et al.*, "Unmanned aerial vehicle scheduling problem for traffic monitoring," *Comput. Ind. Eng.*, vol. 122, pp. 15–23, 2018.
- [27] N. Mathew, S. L. Smith, and S. L. Waslander, "Multirobot rendezvous planning for recharging in persistent tasks," *IEEE Trans. Robot.*, vol. 31, no. 1, pp. 128–142, 2015.
- [28] Y. Ding, W. Luo, and K. Sycara, "Heuristic-based multiple mobile depots route planning for recharging persistent surveillance robots," in *IROS*, 2019, pp. 3308 – 3313.
- [29] K. Yu, A. K. Budhiraja, and P. Tokekar, "Algorithms for routing of unmanned aerial vehicles with mobile recharging stations," in *ICRA*, 2018, pp. 5720–5725.
- [30] J. Diller and Q. Han, "Energy-aware UAV path planning with adaptive speed," in *AAMAS*, 2023.
- [31] P. Maini *et al.*, "Cooperative aerial-ground vehicle route planning with fuel constraints for coverage applications," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 55, no. 6, pp. 3016–3028, 2019.
- [32] E. Arribas, V. Cholvi, and V. Mancuso, "Optimizing UAV resupply scheduling for heterogeneous and persistent aerial service," *IEEE Trans. Robot.*, vol. 39, no. 4, pp. 2639–2653, 2023.
- [33] S. D. Ramchurn, M. Polukarov, A. Farinelli, N. Jennings, and C. Trong, "Coalition formation with spatial and temporal constraints," in *AAMAS*, 2010, pp. 1181–1188.
- [34] W. Gosrich *et al.*, "Multi-robot coordination and cooperation with task precedence relationships," in *ICRA*, 2023, pp. 5800–5806.
- [35] D. Bredström and M. Rönnqvist, "Combined vehicle routing and scheduling with temporal precedence and synchronization constraints," *Eur. J. Oper. Res.*, vol. 191, no. 1, pp. 19–31, 2008.
- [36] E. Feo Flushing, L. M. Gambardella, and G. A. Di Caro, "A mathematical programming approach to collaborative missions with heterogeneous teams," in *IROS*, 2014, pp. 396–403.
- [37] B. Miloradovic, B. Curuklu, M. Ekstrom, and A. V. Papadopoulos, "GMP: A Genetic Mission Planner for Heterogeneous Multirobot System Applications," *IEEE Trans. Cybern.*, vol. 52, no. 10, pp. 10 627–10 638, 2022.
- [38] X. Bai, W. Yan, S. S. Ge, and M. Cao, "An integrated multi-population genetic algorithm for multi-vehicle task assignment in a drift field," *Information Sciences*, vol. 453, pp. 227–238, 2018.
- [39] F. Yan, J. Chu, J. Hu, and X. Zhu, "Cooperative task allocation with simultaneous arrival and resource constraint for multi-UAV using a genetic algorithm," *Expert Syst. Appl.*, vol. 245, p. 123023, 2024.
- [40] S. Liu *et al.*, "Balanced task allocation and collision-free scheduling of multi-robot systems in large spacecraft structure manufacturing," *Robot. Auton. Syst.*, vol. 159, p. 104289, 2023.
- [41] R. Penicka, J. Faigl, M. Saska, and P. Vana, "Data collection planning with non-zero sensing distance for a budget and curvature constrained unmanned aerial vehicle," *Autonomous Robots*, vol. 43, no. 8, pp. 1937–1956, 2019.
- [42] R. Pěnička, J. Faigl, and M. Saska, "Variable Neighborhood Search for the Set Orienteering Problem and its application to other Orienteering Problem variants," *Eur. J. Oper. Res.*, vol. 276, no. 3, pp. 816–825, 2019.
- [43] K. Helsgaun, "An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems," Roskilde University, Tech. Rep., 2017.
- [44] S. Al-Hussaini, J. M. Gregory, and S. K. Gupta, "Generating task reallocation suggestions to handle contingencies in human-supervised multi-robot missions," *IEEE Trans. Autom. Sci. Eng.*, pp. 1–15, 2023.
- [45] M. C. Gombolay, R. J. Wilcox, and J. A. Shah, "Fast scheduling of robot teams performing tasks with temporospatial constraints," *IEEE Trans. Robot.*, vol. 34, no. 1, pp. 220–239, 2018.
- [46] A. Messing, G. Neville, S. Chernova, S. Hutchinson, and H. Ravichandar, "GRSTAPS: Graphically Recursive Simultaneous Task Allocation, Planning, and Scheduling," *Int. J. Rob. Res.*, vol. 41, no. 2, pp. 232–256, 2022.
- [47] Z. Ma, J. Xiong, H. Gong, and X. Wang, "Adaptive Depth Graph Neural Network-based Dynamic Task Allocation for UAV-UGVs Under Complex Environments," *IEEE Trans. Intell. Veh.*, pp. 1–14, 2024.
- [48] M. Limbu, Z. Hu, X. Wang, D. Shishika, and X. Xiao, "Scaling team coordination on graphs with reinforcement learning," in *ICRA*, 2024, pp. 16 538–16 544.
- [49] G. Neville, S. Chernova, and H. Ravichandar, "D-ITAGS: A Dynamic Interleaved Approach to Resilient Task Allocation, Scheduling, and Motion Planning," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 1037–1044, 2023.
- [50] M. Lippi, P. D. Lillo, and A. Marino, "A task allocation framework for human multi-robot collaborative settings," in *ICRA*, 2023, pp. 7614–7620.
- [51] K. Braekers, K. Ramaekers, and I. Van Nieuwenhuysse, "The vehicle routing problem: State of the art classification and review," *Comput. Ind. Eng.*, vol. 99, pp. 300–313, 2016.
- [52] A. Anirudh Sabnis, R. K. Sitaraman, and D. Towsley, "OCCAM: An Optimization Based Approach to Network Inference," *SIGMETRICS Perform. Eval. Rev.*, vol. 46, no. 2, p. 36–38, 2019.
- [53] W. Tan and B. Khoshnevis, "A linearized polynomial mixed integer programming model for the integration of process planning and scheduling," *Journal of Intelligent Manufacturing*, vol. 15, no. 5, pp. 593–605, 2004.
- [54] N. Wilde and J. Alonso-Mora, "Designing heterogeneous robot fleets for task allocation and sequencing," in *MRS*, 2023, pp. 156–162.
- [55] M. Colledanchise and P. Ögren, "How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions, the Subsumption Architecture, and Decision Trees," *IEEE Trans. Robot.*, vol. 33, no. 2, pp. 372–389, 2017.